

Implementable Witness Encryption from Arithmetic Affine Determinant Programs

Lev Soukhanov, Yaroslav Rebenko, Muhammad El Gebali, Mikhail Komarov,
Handan Kilinc-Alper, Michel Abdalla, and Patrick Towa

`{lev, yar, gebali, nemo, handan, michel, patrick}@allocinit.xyz`

`[[alloc] init]`

Witness encryption (WE) for general NP remains challenging to realize with practical efficiency. A natural approach is to combine WE with SNARKs to obtain succinct witnesses; however, embedding general-purpose SNARK verification into existing WE constructions leads to prohibitive overhead. We introduce arithmetic affine determinant programs (AADPs), a variant of affine determinant programs designed to natively support arithmetic constraint systems. This enables direct expression of SNARK verification. To address security challenges in the arithmetic setting, we define projectively safe constraint systems and develop algebraic gadgets that preserve this property. Using these components, we construct a WE scheme for all NP languages based on AADPs, instantiated with a tailored SNARK. Our estimates indicate ciphertext sizes on the order of hundreds of terabytes at 100-bit security, which, to the best of our knowledge, is the most efficient instantiation of WE for general NP.

Table of Contents

1	Introduction.....	3
1.1	Related Work.....	4
1.2	Technical overview for the AADP-based witness encryption.....	5
2	Arithmetic Affine Determinant Programs.....	6
2.1	ADPs.....	6
2.2	Projectively safe constraint systems language.....	7
2.3	AADP generation procedure.....	8
2.4	Witness encryption from ADPs.....	10
3	Cryptanalysis.....	11
3.1	Correlated span attack.....	12
3.2	Necessity of the projective safety condition.....	13
3.3	Parametric rank drop search.....	14
3.4	Linearization attack.....	14
3.5	Security claims.....	16
4	Projectively Safe Gadgets.....	17
4.1	Notation.....	17
4.2	First examples.....	18
4.3	A Proof System for AADPs.....	21
4.4	Finding the extension basis.....	26
4.5	Certification toolbox.....	27
4.6	Pairing computation.....	29
4.7	Poseidon hash.....	33
4.8	Non-native arithmetic.....	34
4.9	Final gadget.....	35
4.10	Cost estimates.....	35

1 Introduction

Witness encryption (WE) [GG⁺13], introduced by Garg et al., is a primitive that allows encrypting a message relative to an NP statement, such that decryption is possible if and only if the statement admits a valid witness. This expressiveness makes WE a natural tool for applications such as advanced forms of access control.

Despite its conceptual simplicity, constructing witness encryption for general NP languages has proven to be extremely challenging. Existing constructions rely on heavy cryptographic machinery such as indistinguishability obfuscation [GG⁺13; BGI⁺01], multilinear maps [GG⁺13; GLW14], or non-standard assumptions [VWW22; Wee22; Tsa22]. They typically offer limited security guarantees or impractical efficiency. As a result, no currently known WE scheme simultaneously achieves generality and practical implementability.

A natural first attempt to improve efficiency is to combine witness encryption with SNARKs: given an NP relation R , one can define a derived language whose statements assert the existence of a valid SNARK proof for R , and use such a proof as the witness. This approach indeed makes witnesses succinct and is formally sound, but it does not resolve the main efficiency bottleneck in existing WE frameworks. In particular, the WE instance must still enforce verification of a general-purpose SNARK, and encoding SNARK verification inside the WE predicate (or the underlying algebraic mechanism) remains prohibitively expensive. Thus, while SNARKs compress witnesses, they do not by themselves reduce the complexity of the computation that the WE scheme must represent. In this paper, we construct a WE scheme that enforces SNARK verification, thereby obtaining a WE scheme for all NP languages while maintaining practically implementable parameters.

Affine Determinant Programs. The notion of *Affine Determinant Programs* (ADPs), formalized in [BIJ⁺20] and implicitly used in several prior works [IK97; IK00; IK02] in context of randomized encodings [AIK07], currently provides one of the most efficient known approaches to witness encryption. In this notion, a boolean function $f : \{0,1\}^n \rightarrow \{0,1\}$ is represented via a set of square matrices $M^0, M^1, \dots, M^n \in \mathbb{F}^{k \times k}$ over a finite field $\mathbb{F}$ and an evaluation function $\text{Eval} : \mathbb{F} \rightarrow \{0,1\}$ such that $f(x) = \text{Eval}(\det(M^0 + \sum_{i=1}^n x_i M^i))$. Typical evaluation functions include testing whether the determinant is zero, nonzero, or a function of its parity.

As shown by Ishai and Kushilevitz [IK02], ADPs can achieve *one-time* simulation-based security for all branching programs. This means that, for a single input x , the evaluated matrix can be simulated given only the value $f(x)$.

While one-time security may suffice for certain applications, this is not the case when an adversary is given the full description of the affine determinant program and is allowed to evaluate it on multiple inputs, including non-binary ones. In the latter case, which is relevant for witness encryption, the determinant becomes a low-degree polynomial that extends the boolean function to the entire field. As shown in [BIJ⁺20], it is often possible to learn this polynomial extension, even for simple functions such as point functions or read-once branching programs.

Since ADPs do not inherently provide reuse security, Bartusek et al. [BIJ⁺20] introduce various randomization techniques such as structured noise and parity-based evaluation to use ADPs as a foundation for obfuscation and witness encryption. In the WE case, Bartusek et al. build witness encryption for the NP-complete vector subset-sum (VSS) problem, with ciphertext sizes scaling as $\Theta(n^3 \cdot |\mathbb{F}|)$, where n denotes the length of the witness. In the VSS problem of dimension d , instances are defined by a matrix-vector pair $(\mathbf{H}, \ell)$ where $\mathbf{H} \in \mathbb{Z}^{d \times n}$ and $\ell \in \mathbb{Z}^d$ and witnesses are vectors $w \in \{0,1\}^n$, such that $\mathbf{H} \cdot w = \ell$ over the integers.

Unfortunately, embedding a general-purpose SNARK verifier whose logic is naturally expressed over arithmetic circuits requires translating arithmetic operations into bit-level constraints as VSS is inherently a combinatorial, bit-level problem. In particular, this translation incurs a massive blowup: a circuit for verifying a general-purpose SNARK can translate into VSS instances where n is in the order of

10^9 , resulting in ciphertext sizes on the order of 10^{27} (for standard field sizes). Consequently, directly composing a SNARK with the original ADP-based framework leads to infeasible ciphertext growth.

Our contributions. In this work, we address this bottleneck by introducing arithmetic affine determinant programs (AADPs), a variant of ADPs designed to natively support arithmetic constraint systems rather than bit-level combinatorial languages. This shift enables us to express the verification of a general-purpose SNARK directly as an arithmetic constraint system, avoiding the blowup incurred by bit decomposition. Our security claim remains heuristic, as in [BIJ⁺20], and we support it via extensive cryptanalysis.

A key technical challenge arises when extending ADPs from bit-level representations to arithmetic constraint systems. In the arithmetic setting, the program may admit unintended evaluations that correspond to projective infinity of the solution space. To address this issue, we introduce the notion of *projectively safe* constraint systems, which ensure that these solutions “at infinity” do not exist. This structural property is crucial for security and plays a central role in both our construction and its analysis.

Building on this foundation, we develop a collection of algebraic gadgets that realize the basic arithmetic operations required for SNARK verification within projectively safe constraint systems. We then design a pairing-based SNARK, in the spirit of Garuda [DMS24], whose verification procedure is carefully structured to be compatible with these gadgets. The resulting construction integrates seamlessly with AADPs and yields, for the first time, an implementable instantiation of ADP-based witness encryption for all NP languages. Our preliminary estimates suggesting ciphertext sizes ≈ 338 TB at 100 bits of security.

1.1 Related Work.

WE, introduced in [GGS⁺13], is implied by indistinguishability obfuscation (iO) [GGS⁺13; GGH⁺13], but appears strictly weaker based on black-box separations [GMM17]. Despite this apparent gap, constructing WE for all NP remains challenging. Current approaches under standard assumptions rely on recent iO constructions [JLS21; JLS22; RVV24], which are far from practical. To achieve more efficient schemes, several direct WE constructions for NP have been proposed based on different assumptions or heuristics, such as evasive LWE [VWW22; Tsa22], the GapMDP problem [BIO⁺20], or ADPs [BIJ⁺20].

Constructions from evasive LWE. Despite recent questions about their security [BÜW24; AMY⁺25; HJL25; DJM⁺25], evasive-LWE-based constructions [VWW22; Tsa22] still remain far from practical.

Consider for instance [VWW22, Construction 6.2], which is restricted to binary circuits using GGH15-style encodings of matrix branching programs [GGH15] and only supports encryption of a single message bit. To estimate the size of its ciphertext ct , let m and n be standard lattice parameters unrelated to our notation and let h denote the number of CNF literals. Since ct can be estimated at $\Theta(h)$ matrices in $\mathbb{Z}_q^{m \times m}$, $|ct| = \Theta(hm^2 \log q)$. As [VWW22, Remark 5.1] requires $\log q = n^\delta = h^2 \lambda$, this adds up to $|ct| = O(h^{2/\delta+5} \lambda^{1/\delta+2})$ by adhering to the LWE choice $m = O(\sqrt{n \log q})$ (omitting w for simplicity). For a standard choice of $\delta = 1/2$, the size of ct would be $O(h^9 \lambda^4)$ in contrast to $\Theta(h^3 \lambda)$ for our construction. This also omits that [VWW22] requires λ ciphertexts for a KEM while ours only requires one.

If we fix $h \approx 10^9$ for the binary arithmetization of a SNARK verifier and set $\lambda = 100$ bits for a fair comparison, a direct substitution would yield 10^{89} bits versus $\sim 10^{15}$ bits for our scheme.

Constructions from GapMDP problem. [BIO⁺20] gives a WE construction from the GapMDP problem. It is provably secure only if its approximation factor β is super-logarithmic in the code length and if the field is of super-polynomial size, say q (at least 2^{2^λ} for the discrete log problem to be hard in a generic group). If we assume the construction in [BIO⁺20] is secure with constant β (note that GapMDP problem is not known to be NP hard for such β), known NP reductions (for example, [AK14]) to GapMDP would result in a code size at least $\text{poly}(q)$ and ciphertexts scaling exponentially in λ .

ADP-based Constructions. Among the direct approaches for NP, ADPs [BIJ⁺20] provide one of the most efficient heuristic approaches to WE for NP. However, as mentioned earlier, the resulting ciphertexts become prohibitively large when instantiated with SNARK-verifiable predicates.

Constructions for restricted classes. Several works have explored WE for restricted classes of languages. In particular, when the underlying relation admits a linear pairing-based verifier, one can obtain efficient WE constructions [BC16; BL20; CFK24; FHAS24; GKP⁺24]. These techniques, however, do not extend to general-purpose SNARKs such as Groth16 [Gro16], whose verifier is quadratic. More recently, [GHK⁺25] provided a unifying framework for such linear-verifier constructions via succinct linearly-verifiable arguments.

Our work. Our work builds on the ADP framework and addresses the inefficiency that arises when embedding general SNARK verification, enabling an *implementable* WE construction for all NP while retaining the heuristic security model of ADPs.

1.2 Technical overview for the AADP-based witness encryption

Witness encryption from ADPs [BIJ⁺20]. In [BIJ⁺20], Bartusek et al. use ADPs to build a new framework for witness encryption for the NP-complete vector subset-sum (VSS) problem. In the VSS problem of dimension d , instances are defined by a matrix-vector pair $(\mathbf{H}, \ell)$ where $\mathbf{H} \in \mathbb{Z}^{d \times n}$ and $\ell \in \mathbb{Z}^d$ and instances are vectors $w \in \{0, 1\}^n$, such that $\mathbf{H} \cdot w = \ell$ over the integers. One can recover the standard subset problem by setting $d = 1$.

Let a width- k ADP M over $\mathbb{F}_p$ refer to a tuple of $n + 1$ matrices $(M^0, M^1, \dots, M^n)$, with $M^i \in \mathbb{F}_p^{k \times k}$. As shown in [BIJ⁺20], one can easily encode a dimension $d = 1$ instance (h, ℓ) of the VSS problem as an ADP $M_{h, \ell}$ of width k by sampling a uniformly random matrix $\mathbf{R} \in \mathbb{F}_p^{k \times k}$ and setting $M^0 = -\ell \mathbf{R}$ and $M^i = h_i \mathbf{R}$ for $i \in [n]$, where $h = [h_1, \dots, h_n]$. Such an ADP would accept an input $w \in \{0, 1\}^n$ if and only if $\det(M_{h, \ell}(w)) = 0$, which implies with overwhelming probability over the choice of $\mathbf{R}$ that w is a valid witness.

To construct a witness encryption from the VSS-encoded ADP $M_{h, \ell}$, a natural approach would be to encrypt a message bit b by adding $b\mathbf{S}$ to M^0 for a uniformly random matrix $\mathbf{S} \in \mathbb{F}_p^{k \times k}$. In this context, for any valid witness w , it is evident that $\det(M(w)) = 0$ if and only if $b = 0$. However, as highlighted in [BIJ⁺20], this method is entirely insecure since an attacker could easily find a vector w with arbitrary field elements such that $h \cdot w = \ell$, which would not contradict the hardness of the subset problem.

To circumvent the issue of invalid inputs, [BIJ⁺20] combines $M_{h, \ell}$ with a second All-Accept ADP M_{AA} , so that the rank of $M_{AA}(w)$ is deficient for any input $w \in \{0, 1\}^n$ and full otherwise. Consequently, any invalid input to the ADP $M_{h, \ell} + M_{AA}$ will result in full-rank matrices with a high probability, thereby effectively concealing the bit b that is being encrypted.

To build M_{AA} , [BIJ⁺20] employs a formula encoding scheme for the boolean formula $f(x_1, \dots, x_n) = (x_1 \vee \neg x_1) \wedge \dots \wedge (x_n \vee \neg x_n)$. In this scheme, positive and negative input wires are represented as width-1 ADPs and the ADP for OR and AND gates is constructed recursively by combining the ADPs associated with their respective inputs. Let $M_1(w)$ and $M_2(w)$ be two ADPs of dimensions k_1 and k_2 over inputs $w \in \{0, 1\}^n$ and let $k = k_1 + k_2$. In the case of AND gates, the new ADP for $M_{AND}(w)$ is generated by first creating a matrix of dimension $k \times k$ with $M_1(w)$ and $M_2(w)$ placed in the diagonal and zeros on the top right and bottom left and then applying left and right randomization using matrices of dimensions $(k - 1) \times k$ and $k \times (k - 1)$, respectively. In the case of OR gates, $M_{OR}(w)$ is generated by first creating a matrix of dimension $k \times k$ with $M_f(w)$ and $M_g(w)$ placed in the diagonal, a uniform ADP on the top right, and zeros on the bottom left, and then applying left and right randomization using matrices of dimensions $k \times k$. This combination ensures that the output ADP M will be rank deficient by 1 only on valid inputs w . Specifically, for an OR gate, the output ADP M_{OR} will be rank deficient by 1 on an input w if at least one of the input ADPs M_1 and M_2 is rank deficient on w and full rank otherwise.

Similarly, for an AND gate, the output ADP M_{AND} will be rank deficient by 1 on an input w only if both input ADPs M_1 and M_2 are rank deficient on w and full rank otherwise.

Witness encryption from AADPs. While the ADP-based witness encryption construction for the VSS problem proposed in [BIJ⁺20] appears significantly more practical than previous approaches in the literature, the resulting scheme is still not efficient enough to embed a general-purpose SNARK verifier. As mentioned earlier, the resulting ciphertext would still be unrealistically large in this case.

To get around the problem of embedding general-purpose SNARKs in VSS-based ADPs, we first introduce arithmetic affine determinant programs (AADPs) in Section 2.1, which is a variant of ADPs that natively supports arithmetic constraint systems. Unlike ADPs, which only accept inputs in $\{0, 1\}^n$, AADPs accept inputs from an algebraic closure $\overline{\mathbb{F}}$ of a field $\mathbb{F}$.

The language for which we build a witness-encryption scheme is defined by a class of arithmetic constraint systems akin to R1CS. It is parametrized by matrices $A, B, C, D \in F^{m \times (n+1)}$ describing m quadratic constraints over the variables $\mathbf{x} = (x_0, x_1, \dots, x_n)$ such that $(A\mathbf{x}) \circ (B\mathbf{x}) = (C\mathbf{x}) \circ (D\mathbf{x})$, where $\circ$ denotes the Hadamard (element-wise) product. For the witness-encryption scheme to be secure, the constraint systems must satisfy the notion of projective safety (defined in Section 2.2) which essentially guarantees that any valid non-trivial solution to the constraint system can be mapped to an affine solution $\mathbf{x}$ such that $x_0 = 1$. The satisfiability of such constraint systems is still clearly NP complete.

The construction of an AADP M from a projectively safe constraint system (A, B, C, D) shares a similar intuition with the formula-based construction of the All-Accept ADP M_{AA} in [BIJ⁺20]. Specifically, when given a constraint system (A, B, C, D) of dimension $m \times (n + 1)$, the resulting AADP M is encoded as a vector of $n + 1$ square matrices $(M^0, \dots, M^n)$ in $\mathbb{F}^{(2m+1) \times (2m+1)}$. This encoding ensures that, for any valid input $\mathbf{x}$ that satisfies the constraint $(A\mathbf{x}) \circ (B\mathbf{x}) = (C\mathbf{x}) \circ (D\mathbf{x})$, the resulting matrix $M(\mathbf{x}) = \sum_{i=0}^n x_i M^i$ will have rank $2m$ or be full rank otherwise.

To achieve this goal, as outlined in Section 2.3, $M(\mathbf{x})$ will be computed as the sum of m matrices $\mathbf{M}_j(\mathbf{x})$ whose ranks are equal to 2 if the j -th constraint in (A, B, C, D) is satisfied and 4 otherwise. Consequently, $M(\mathbf{x})$ will be rank deficient if and only if $\mathbf{x}$ is a valid solution to (A, B, C, D) .

Compared to the ADP construction [BIJ⁺20], there is now a single program that attests to the satisfiability of an NP-complete problem with witness coefficients in the whole field instead of a first program handling linear relations and a second one to enforce that the provided witness is Boolean. The original construction could also be optimized to only communicate linear combinations of matrices M^i that satisfy VSS linear constraints. After this optimization, AADPs become just a version of ADPs with a different kind of elementary non-linear gate.

Finally, to encrypt a message $\text{msg} \in \mathbb{F}$ with respect to (A, B, C, D) , we simply add msg to the bottom-right entry of the matrix M^0 of the corresponding AADP M . Like the original ADP-based construction, the security of our construction is supported via extensive cryptanalysis but remains heuristic.

2 Arithmetic Affine Determinant Programs

2.1 ADPs

First we give a short review of the ADP construction [BIJ⁺20]

Definition 1. An affine determinant program $ADP : \{0, 1\}^n \rightarrow \{0, 1\}$ is specified by square matrices $M^0, \dots, M^n$ over a field $\mathbb{F}$ and a function $\text{Eval} : \mathbb{F} \rightarrow \{0, 1\}$. It is evaluated on input $\mathbf{x} \in \{0, 1\}^n$ by computing

$$\text{Eval}\left(\det\left(M^0 + \sum_{i=1}^n x_i M^i\right)\right)$$

We modify this definition to accept inputs from an algebraic closure $\overline{\mathbb{F}}$. Changes are *highlighted*.

Definition 2. An arithmetic affine determinant program $AADP : \mathbb{F}^n \rightarrow \{0, 1\}$ is specified by a collection of square matrices $M^0, \dots, M^n$ over a field $\mathbb{F}$. It is evaluated on a **nonzero** input $\mathbf{x} \in \mathbb{F}^n$ by computing

$$\text{Eval}_0 \left(\det \left(M^0 + \sum_{i=1}^n x_i M^i \right) \right)$$

Our main objective is to adapt ADPs for arithmetic circuits and, ideally, to avoid decomposition into bits. The changes are motivated as follows:

1. Given that the determinant defines a polynomial defined over $\mathbb{F}$, the inputs are necessarily from $\mathbb{F}$ as it is impossible to prevent the evaluation of an ADP on elements of an algebraic closure.
2. **Eval** mapping is specified to $\text{Eval}_0(v) = \text{if } (v = 0) \{1\} \text{ else } \{0\}$ because it is the only case relevant to the witness encryption construction.

A significant effort in the original paper is spent on the “rejection of invalid inputs” property, which enforces $(x_1, \dots, x_n) \in \{0, 1\}^n$. In our case, we enforce a different quadratic nonlinear relation.

In what follows, we will always assume that $\mathbb{F}$ is cryptographically large. While it is not strictly needed in the original paper, it significantly simplifies our analysis and our main application uses a large field anyway.

In the next section, we will specify the class of constraint systems for which we define our arithmetic ADP programs.

2.2 Projectively safe constraint systems language

We define a class of arithmetic constraint systems, called *projectively safe*, which will serve as the NP language for our AADP construction and ensure sound behavior under projective evaluation.

Let $\mathbb{F}$ be a field, and let $A, B, C, D \in \mathbb{F}^{m \times (n+1)}$. We denote by $A_i^k \in \mathbb{F}$ the entry of A in row i and column k , and analogously for B_i^k , C_i^k , and D_i^k . We consider systems of quadratic constraints over variables $(x_0, x_1, \dots, x_n)$ of the form

$$(i) : \left(\sum_{k=0}^n A_i^k x_k \right) \cdot \left(\sum_{k=0}^n B_i^k x_k \right) = \left(\sum_{k=0}^n C_i^k x_k \right) \cdot \left(\sum_{k=0}^n D_i^k x_k \right) \quad (1)$$

An *affine solution* is an assignment $(x_0, x_1, \dots, x_n)$ with $x_0 = 1$ and $(x_1, \dots, x_n) \in \mathbb{F}^n$ satisfying all constraints Eq. (1).

We will also consider projective solutions - i.e. vectors $(x_0, \dots, x_n) \neq 0$ satisfying this constraint system but without the requirement $x_0 = 1$. Any projective solution can be rescaled to a regular affine solution if $x_0 \neq 0$. Projective solutions with $x_0 = 0$ will be referred to as *solutions at infinity*.

Remark 1. A projective solution $(x_0, \dots, x_n)$ with $x_0 \neq 0$ uniquely determines an affine solution by normalization, namely

$$(x_0, \dots, x_n) \mapsto \left(1, \frac{x_1}{x_0}, \dots, \frac{x_n}{x_0} \right).$$

Thus, in this case, knowledge of a projective solution is equivalent to knowledge of a valid affine solution to the constraint system.

In contrast, a projective solution with $x_0 = 0$ does *not* correspond to any affine solution and therefore represents additional algebraic structure that is not part of the intended NP language. Such solutions at infinity cannot be ruled out by affine normalization and must be explicitly excluded as we define next.

Definition 3 (Projective safety). Fix an ordering of variables

$$x_0 \prec x_1 \prec \dots \prec x_n.$$

An arithmetic constraint system of the form [Eq. \(1\)](#) is called projectively safe if for every variable x_i , with $1 \leq i \leq n$, there exists an index $j_i \in \{1, \dots, m\}$ such that the corresponding constraint (j_i) can be written as

$$x_i^2 = a(x_0, \dots, x_{i-1}) \cdot b(x_0, \dots, x_n),$$

for some linear forms a, b over $\mathbb{F}$. Such a constraint (j_i) is called a **certificate** for the variable x_i .

Intuitively, projective safety guarantees that every projective solution with $x_0 = 0$ is trivial, so that all nontrivial projective solutions uniquely correspond to valid affine solutions as we can see in [Lemma 1](#).

Lemma 1. A projectively safe constraint system has no nonzero projective solutions at infinity.

Proof. Let $(x_0, \dots, x_n)$ be a projective solution at infinity ($x_0 = 0$). By projective safety, for each $i \geq 1$ there exists a certificate constraint (j_i) of the form $x_i^2 = a(x_0, \dots, x_{i-1}) \cdot b(x_0, \dots, x_n)$. If $x_0 = \dots = x_{i-1} = 0$, then the right-hand side vanishes, implying $x_i^2 = 0$ and hence $x_i = 0$. By induction on i , this yields $x_1 = \dots = x_n = 0$, contradicting the requirement that a projective solution is nonzero. $\square$

The motivation for projective safety is that AADPs or ADPs inherently allow evaluation of the matrix polynomial $\sum_{i=0}^n x_i M^i$ on *arbitrary* inputs $(x_0, \dots, x_n)$. In particular, there is no mechanism in the AADP or ADP framework to prevent an adversary from evaluating this expression on projective inputs with $x_0 = 0$.

If such evaluations admit nontrivial solutions, they yield low-rank matrices that do not correspond to any valid affine witness and may therefore leak additional algebraic structure (See [Section 3.2](#)). Projective safety rules out this possibility by ensuring that all projective solutions with $x_0 = 0$ are trivial.

It is worth noting that the original ADP construction of [\[BIJ⁺20\]](#) avoids this issue implicitly: bit-checked constraint systems are automatically projectively safe, since the bit-check constraints force all variables to vanish whenever $x_0 = 0$ as stated in [Lemma 2](#), whose proof is straightforward and omitted.

Lemma 2. Assume that every variable x_i is bitchecked, i.e. is subject to a constraint $x_i^2 = x_0 \cdot x_i$. Then, this system is projectively safe.

While bit-checking all variables trivially guarantees projective safety, enforcing such constraints in the ADP framework is prohibitively expensive as each additional nonlinear constraint contributes substantially to the size of the resulting determinant program and hence to the ciphertext. We therefore allow variables to range over $\overline{\mathbb{F}}$ and instead enforce projective safety at the level of the constraint system, which rules out nontrivial solutions at infinity without requiring per-variable bit-checking. This approach, combined with the gadget optimizations in [Section 4](#), yields substantially more efficient constructions.

2.3 AADP generation procedure

We define an algorithm $\text{AADP.Gen} : (A, B, C, D) \mapsto M$ that takes as an input projectively safe constraint system (A, B, C, D) with n variables and m constraints and outputs an arithmetic affine determinant program M .

We will use the notation $\leftarrow$ to denote a uniform random choice in a set and we will work with linear combinations in variables $x_0, \dots, x_n$. In particular, our M will be a linear combination-valued matrix of dimensions $(2m+1) \times (2m+1)$ which corresponds to $M(x) = \sum_{i=0}^n x_i M^i$ in standard ADP notation.

Algorithm 1 Algorithm AADP.Gen

```

for  $0 \leq i < m$  do
   $a_i(x) \leftarrow \sum_{j=0}^n A_i^j x_j$ 
   $\triangleright$  repeat for  $b, c, d$  and  $B, C, D$  respectively  $\triangleleft$ 
   $\xi_i(x) \leftarrow \sum_{j=0}^n \xi_i^j x_j$  where for  $0 \leq j \leq n : \xi_i^j \leftarrow \mathbb{F}$ 
   $U_i(x) \leftarrow \begin{pmatrix} a_i(x) & c_i(x) - \xi_i(x) & 0 & \\ d_i(x) & b_i(x) & 0 & \xi_i(x) \\ 0 & 0 & b_i(x) & c_i(x) \\ 0 & 0 & d_i(x) & a_i(x) \end{pmatrix}$ 
   $L_i \leftarrow [\mathbb{F}]^{(2m+1) \times 4}$ 
   $R_i \leftarrow [\mathbb{F}]^{4 \times (2m+1)}$ 
   $M_i(x) \leftarrow L_i \cdot U_i(x) \cdot R_i$ 
end for
return  $M(x) = \sum_{i=0}^{m-1} M_i(x)$ 

```

Lemma 3. For a given x and random setup, $\text{rank}(U(x)) = \begin{cases} 4 & ab \neq cd \\ 2 & ab = cd \end{cases}$ except with negligible probability.

Proof.

- If $ab \neq cd$: $\det(U) = (ab - cd)^2 \neq 0$ hence the matrix has a full rank
- If $ab = cd$: bottom two rows $\begin{bmatrix} 0 & 0 & b & c \\ 0 & 0 & d & a \end{bmatrix}$ are linearly dependent and can be rewritten using Gauss elimination as $\begin{bmatrix} 0 & 0 & v & u \\ 0 & 0 & 0 & 0 \end{bmatrix}$ and the same can be done for the two left columns. Substituting the result back into U we obtain a minor $\tilde{U} = \begin{bmatrix} u - \xi & 0 \\ v & 0 & \xi \\ 0 & v & u \end{bmatrix}$ with $\det(\tilde{U}) = 0$ and $\text{rank}(U) = \text{rank}(\tilde{U}) \leq 2$. Except with negligible probability we have $\xi \neq 0$ and so rank can not be less than 2 due to a submatrix $\begin{bmatrix} -\xi & 0 \\ 0 & \xi \end{bmatrix}$ having nonzero determinant.

□

Theorem 1. For a constraint system (A, B, C, D) , a fixed projective input vector $\tilde{x} = (x_0, \dots, x_n)$, and a random setup

- If $\tilde{x}$ is a valid projective solution of the constraint system, $\det M(\tilde{x}) = 0$ and M is statistically indistinguishable from a random matrix of rank $2m$.
- If $\tilde{x}$ is not a valid projective solution, M is statistically indistinguishable from a random $2m+1$ (full) rank matrix.

This property is called “one-time security” in [BIJ⁺20].

Proof. Consider a matrix $M_i(x) = L_i U_i(x) R_i$ produced on the i -th step of the algorithm AADP.Gen (Algorithm 1). By Lemma 3 if a constraint (i) does not hold, the matrix $U_i(x)$ has a rank 4 and if it does hold, it has rank 2 except with negligible probability.

The distribution of matrices $M_i(x)$ is symmetric under the following changes:

$$\begin{aligned} L_i &\mapsto X L_i \\ R_i &\mapsto R_i Y \end{aligned} \quad \text{and hence} \quad M_i(x) \mapsto X M_i(x) Y$$

where X, Y are random invertible $2m + 1 \times 2m + 1$ matrices.

This group of transforms acts transitively on the space of matrices of a fixed rank. This means that $M_i(x)$ is statistically indistinguishable from a random rank 2 matrix if a constraint (i) held, and random rank 4 matrix if it didn't.

The sum of m random equidistributed rank 2 matrices is a matrix of rank $2m$ except with negligible probability. On the contrary, if at least one of the constraints didn't hold, the sum is a full-rank matrix except with negligible probability.

To see that the sum is statistically indistinguishable from a random matrix (of ranks $2m$ and $2m + 1$ respectively), consider once again an action that simultaneously changes each $L_i \mapsto X L_i$ and $R_i \mapsto R_i Y$. This transforms $M(x) \mapsto X M(x) Y$ which is the transitive action. Therefore, $M(x)$ is equidistributed. $\square$

2.4 Witness encryption from ADPs

Witness encryption reminder. Witness encryption, introduced in [GG⁺13] is a generalization of public key encryption for arbitrary NP languages. Specifically, given a message space Msg , an NP-language L and corresponding witness relation $R(x, w)$, witness encryption with security parameter 1^λ is a pair of PPT algorithms $\text{Enc}(1^\lambda, x, M)$ and $\text{Dec}(\text{ct}, w)$, such that

- **Correctness:** for all messages $\text{msg} \in \text{Msg}$, $x \in \mathcal{L}$ and w such that $(x, w) \in R$ holds

$$\Pr_{\text{Dec}(\text{Enc}(1^\lambda, x, M), w) = M} [=] 1 - \text{negl}(\lambda)$$

i.e., x plays the role of a public key and w plays the role of a corresponding private key.

- **Extractable security:** for any PPT adversary $\mathcal{A}$ and poly q there exists a PPT extractor $\mathcal{E}$ and poly p such that for all auxiliary inputs z and for all $x \in \{0, 1\}^*$

$$\Pr \left[\begin{array}{l} \text{msg} \leftarrow \text{Msg} \\ \text{ct} \leftarrow \text{Enc}(1^\lambda, x, M) \end{array} : \mathcal{A}(\text{ct}, x, z) = M \right] \geq \frac{1}{|\text{Msg}|} + \frac{1}{q(|x|)} \quad (2)$$

$$\implies \Pr_{\mathcal{E}(x, z) = w : (x, w) \in R} [\geq] \frac{1}{p(|x|)} \quad (3)$$

Or informally: if there is an adversary that can reliably decrypt the message using some knowledge z there is an extractor that can use the same knowledge to solve the instance.

We will omit the 1^λ parameter of Enc .

Witness encryption from AADPs. Now we provide a Witness Encryption construction for AADPs. It is an adaptation of the construction in [BIJ⁺20, Section 6.1.3] to the large field setting.

Definition 4. Define an algorithm

$$\text{WE.Enc} : ((A, B, C, D), \text{msg}) \mapsto \widehat{M}$$

which takes as an input a projectively safe constraint system (A, B, C, D) and a message $\text{msg} \in \mathbb{F}$ and outputs an (arithmetic ADP-formatted) ciphertext $\widehat{M}$ and a corresponding algorithm

$$\text{WE.Dec} : (s, \widehat{M}) \mapsto \text{msg}$$

which takes as input an affine solution $s = (1, s_1, \dots, s_n)$ to the constraint system and a ciphertext $\widehat{M}$ and outputs the encrypted message **msg**:

Note 1. There is an unfortunate collision of notation because the symbols $(x_0, \dots, x_n)$ denote formal variables. Therefore, we denote the affine solution with different symbols $s = (1, s_1, \dots, s_n)$, implying that it is an evaluation in $x_0 \leftarrow 1, x_i \leftarrow s_i$.

Algorithm 2 Algorithm WE.Enc

$M(x) \leftarrow \text{AADP.Gen}((A, B, C, D))$
 $\widehat{M}(x) \leftarrow M(x) + \text{msg} \cdot x_0 \cdot \underbrace{[0, \dots, 0, 1]^T}_{2m \text{ times}} \cdot \underbrace{[0, \dots, 0, 1]}_{2m \text{ times}}$
return $\widehat{M}(x)$

The encryption algorithm just produces an arithmetic ADP and adds a message to a single cell in a matrix M^0 .

Algorithm 3 Algorithm WE.Dec

$E \leftarrow \widehat{M}(s)$
Solve in t : $\det(E - t \cdot \underbrace{[0, \dots, 0, 1]^T}_{2m \text{ times}} \cdot \underbrace{[0, \dots, 0, 1]}_{2m \text{ times}}) = 0$
return $\text{msg} = t$

The decryption algorithm computes a determinant treating **msg** as an indeterminate variable, and then solves a linear equation in it. Compared to the original method in [BIJ⁺20, Section 6.1.3], we do not use any additional columns because our field is cryptographically large which implies that considered linear equation has nonzero coefficient of t except with negligible probability, so the decryption procedure doesn't fail.

Lemma 4. WE.Dec is correct: i.e. it successfully decrypts the output of WE.Enc.

Proof. Fix any valid affine input $s = (1, s_1, \dots, s_n)$ and let $E \leftarrow \widehat{M}(s)$ be the evaluation of the ciphertext ADP at s .

Topleft $2m \times 2m$ minor of the matrix E has nonzero determinant except with negligible probability. This is clear because the matrix $M(s)$ has rank $2m$ and (for a fixed x and varying setup parameters) is equidistributed in the space of all matrices of rank $2m$ by Theorem 1.

Hence, the linear equation in Algorithm 3 has a nonzero coefficient before t and therefore a unique solution.

Now, it remains to see that original encrypted **msg** is indeed a solution. This just means that $\det M(s) = 0$ which is true by Theorem 1. $\square$

3 Cryptanalysis

Neither AADPs nor original ADPs have a reduction to a well-established security assumption, so we need to come up with and analyze specific attacks on our scheme.

3.1 Correlated span attack

Our starting point is the correlated span attack (see [BIJ⁺20, Section 7.1.1] for the original description). It is a randomness recovery attack that requires the adversary to know multiple solutions of the original statement. However, it also would work if the adversary was able to sample a lot of projective solutions at infinity: which is a main motivation behind our notion of projective safety.

We give here a generalized version of the correlated span attack adapted to our setting. We assume the attacker has access to multiple projective solutions to the constraint system.

The attack is heuristic - in the description below, we will assume that these solutions are generic in a sense that they have some nonzero values on each gate and various linear systems formed from them have an expected dimensions of solution spaces. In the cases where an attacker can only sample solutions from some linear subspace it might be harder to mount the attack.

Consider a solution w to a constraint system that corresponds to AADP program M . As $\det(M(w)) = 0$, there is a kernel vector $M(w) \cdot \alpha_w = 0$. If it is unique up to rescaling (which happens except with negligible probability), then $\forall i : U_i(w)R_i\alpha_w = 0$. This happens because rank of each of these systems is 2 and there are m conditions in total, so they do have a unique solution vector. This common vector also necessarily lies in a 1-dimensional kernel of $M(w)$, so it must be α_w .

Consider the i -th gate $a_i b_i = c_i d_i$. Denote the rows of the matrix R as $R_i^1, R_i^2, R_i^3, R_i^4$. Then, for $(b_i(w), c_i(w)) \neq (0, 0)$ the third row of the matrix U_i gives us the following ξ -independent equation:

$$b_i(w)R_i^3\alpha_w + c_i(w)R_i^4\alpha_w = 0$$

for each i .

Treating R_i^3, R_i^4 as variables and sampling $2m$ independent pairs (w, α_w) , we obtain enough linear equations to find all the coefficients of R_i^3 and R_i^4 .

Analogously, starting from a dual kernel $\beta_w M(w) = 0$ one can recover the first two columns L_i^1, L_i^2 of the L -matrices.

Lemma 5. Let w be a valid projective solution, v and u any vector and covector satisfying

$$(b_i(w)R_i^3 + c_i(w)R_i^4)v = 0 \quad u(c_i(w)L_i^1 + b_i(w)L_i^2) = 0$$

Then, a term $uL_iU_i(w)R_iv = 0$

Proof. Explicitly write down $uL_iU_i(w)R_iv$ and simplify it using $a_i(w)b_i(w) = c_i(w)d_i(w)$. We will avoid writing evaluation in w to reduce the visual clutter:

$$\begin{pmatrix} uL_i^1, uL_i^2, uL_j^3, uL_j^4 \end{pmatrix} \cdot \begin{pmatrix} a_i & c_i & -\xi_i & 0 \\ d_i & b_i & 0 & \xi_i \\ 0 & 0 & b_i & c_i \\ 0 & 0 & d_i & a_i \end{pmatrix} \cdot \begin{pmatrix} R_i^1 v \\ R_i^2 v \\ R_i^3 v \\ R_i^4 v \end{pmatrix} \quad (4)$$

$$= \begin{pmatrix} uL_i^1, uL_i^2, uL_j^3, uL_j^4 \end{pmatrix} \cdot \begin{pmatrix} a_i R_i^1 v + c_i R_i^2 v - \xi_i R_i^3 v \\ d_i R_i^1 v + b_i R_i^2 v + \xi_i R_i^4 v \\ 0 \\ 0 \end{pmatrix} \quad (5)$$

$$= a_i(uL_i^1 R_i^1 v) + d_i(uL_i^2 R_i^1 v) + c_i(uL_i^1 R_i^2 v) + b_i(uL_i^2 R_i^2 v) + \xi(uL_i^2 R_i^4 v - uL_i^1 R_i^3 v) \quad (6)$$

Because of our assumption $b_i(R_i^3 v) + c_i(R_i^4 v) = 0$ we can express $R_i^3 v = c_i s$, $R_i^4 v = -b_i s$ for some s and similarly $uL_i^1 = b_i t$, $uL_i^2 = -c_i t$. Substituting this yields:
 $\dots = (a_i b_i t - d_i c_i t)R_i^1 v + (c_i b_i - b_i c_i)R_i^2 v + \xi t s(c_i b_i - b_i c_i) = 0$ □

This structure can be exploited to determine the remaining parts of the matrices, i.e. columns L_i^3, L_i^4 and rows R_i^1, R_i^2 . In order to do that, pick a vector v and a covector u satisfying the following conditions:

$$\forall j \neq i : (b_j(w)R_j^3 + c_j(w)R_j^4)v = 0$$

$$\forall j \neq i : u(c_j(w)L_j^1 + b_j(w)L_j^2) = 0$$

$$R_i^3v = 0 \quad R_i^4v = 0$$

Now, compute the value $uM(w)v = \sum_j uL_jU_j(w)R_jv$. By [Lemma 5](#) only the i -th term survives. Writing it down explicitly gives:

$$uM(w)v = uL_iU_i(w)R_iv = \begin{pmatrix} uL_i^1, uL_i^2, uL_i^3, uL_i^4 \end{pmatrix} \cdot \begin{pmatrix} a_iR_i^1v + d_iR_i^2v \\ c_iR_i^1v + b_iR_i^2v \\ 0 \\ 0 \end{pmatrix} =$$

$$(a_iuL_i^1 + c_iuL_i^2)R_i^1v + (d_iuL_i^1 + b_iuL_i^2)R_i^2v$$

By this point, all the values L_i^1, L_i^2 are already found, so this represents an inhomogeneous linear equation on the values of the matrices R_i^1, R_i^2 . Sampling enough equations of this form, we fully recover values of R . Recovery of L -values is done analogously.

Now, it remains to only recover ξ -values. Notice under conditions that values of L_i and R_i are already known, any expression of the form $uM(w)v$ is a linear equation on values of ξ . To make this process more efficient, we can use only such pairs u, v that satisfy the system

$$\forall j \neq i : (b_j(w)R_j^3 + c_j(w)R_j^4)v = 0$$

$$\forall j \neq i : u(c_j(w)L_j^1 + b_j(w)L_j^2) = 0$$

which ensures that only coefficients of a single ξ_i participate in the equation.

Asymptotic complexity. Each phase of the attack requires sampling $O(m)$ solutions and $O(m^4)$ field operations. A single step where it is not directly obvious is sampling $O(m)$ per each of m gates) vectors u and v for each witness sample w in the second phase of the attack. To see that, observe that for each witness sample all the vectors v are taken from the common subspace of a linear system $\forall j : (b_j(w)R_j^3 + c_j(w)R_j^4)v = 0$, which can be found in $O(m^3)$.

This is somewhat worse than the original correlated span attack described in [\[BIJ⁺20\]](#) which runs in $O(m^3)$ time, but our version has the significant advantage of being mountable in any case where many (generic enough) solutions are known. It is also likely that in case an additional structure is known on the solution space this attack can be significantly sped up.

3.2 Necessity of the projective safety condition

Due to the existence of the correlated span attack, projectively unsafe circuits are typically non-secure. An attacker can ignore the existence of the matrix M^0 and run the correlated span attack for the solutions at infinity to learn all the values of L_i, R_i and ξ_i^j -s for $j > 0$. It can then compute ξ_i^0 and msg by directly running the WE.Enc algorithm in indeterminates and solving the resulting system of linear equations.

3.3 Parametric rank drop search

In an older version of our scheme, we opted to use two different kinds of U-matrices - the standard matrix that checks the gate $ab = cd$ and the bitcheck matrix $\begin{pmatrix} b & \xi \\ 0 & x_0 - b \end{pmatrix}$ to specifically check that $b^2 = b$. This optimization led to the following vulnerability: given a pair of bit variables x_i, x_j that do not participate in same gates, consider the one-parametric family $M^i + tM^j$ corresponding to input $x(t)$ such that $x_i(t) = 1, x_j(t) = t, x_{k \neq i, j}(t) = 0$.

For such input family, exactly two bitcheck constraints are broken for each t . Consider an additional multiplication gate $(k) : ab = cd$ such that linear forms a, b, c, d do not include x_i, x_j . For such a projective input it is totally zero and its corresponding matrix U_k only contains $\pm \xi_k(x(t))$. For some values of t , the value $\xi_k(x(t))$ will vanish, forcing the rank of U_k to become 0 instead of expected 2. These situations can be detected as (some of the roots) of the characteristic polynomial $\chi(t) = \det(M^i + tM^j)$.

This seems to lead to at least partial randomness recovery attack.

Details. Consider a family $\mathcal{X}$ a k -dimensional family of projective solutions at infinity. To show why parametric rank drop search attacks do not apply to our scheme, we show that a generic element of this family breaks at least $k + 1$ constraints. Because the condition in which $k + 1$ different constraints have their ξ -terms vanishing has codimension $k + 1$ these situations do not, generically, occur in this family.

Lemma 6. Let $\mathcal{X} \hookrightarrow \mathbb{P}^{n-1} = \mathbb{P}(x_1 : \dots : x_n)$ be a k -dimensional algebraic variety. A generic $x \in \mathcal{X}$ breaks at least $k + 1$ constraints.

Proof. Let a generic element of this family pass all but $\ell + 1$ certifying constraints corresponding to the variables $x_{i_0}, \dots, x_{i_\ell}$ (and also break any number of non-certifying ones). Consider the space of solutions $\mathcal{Y}$ to our system without certifying constraints corresponding to these variables (i.e., constraints $j_{i_s} : x_{i_s}^2 = \dots$ are allowed to be broken). Obviously, $\mathcal{X} \subset \mathcal{Y}$.

On the other hand, $\mathcal{Y}$ is embedded into a sequence of finite coverings of the coordinate plane $\mathbb{P}(x_{i_0} : \dots : x_{i_\ell})$, corresponding to certifying constraints $(j) : x_j^2 = \dots$ for all $j \notin (i_0, \dots, i_\ell)$. This means that $\mathcal{Y}$ and hence $\mathcal{X}$ is not more than ℓ -dimensional, implying $k \leq \ell$. $\square$

This justifies why this attack does not seem possible in our version of the protocol.

3.4 Linearization attack

We make a best-effort attempt at launching a linearization attack analogous to the one described in [BIJ⁺20, Section 7.1.2]. Similar to the original paper, we do not actually manage to mount it, but we both show that if it exists, then it has infeasible runtime for practically relevant instances, and there is a mitigation from a theoretical standpoint.

The attack in the original paper was possible when the subset-sum instance was sparse and as such one could isolate some B_i matrices in the clear. The analog for AADP is the existence of inputs x such that they are nonzero only on a small amount of quadratic constraints.

The goal of the attack is to recover the random entries of matrices L_i and R_i used to generate the program. To do so, the idea is to exhibit linearly independent polynomial equations that these entries satisfy. Given enough equations, Kipnis and Shamir's relinearization technique [KS99] could then be used to recover these random entries, treating them as random variables. These equations stem from the construction and their coefficients are determined by public values. More precisely, the major part of these equations follow from the fact that a matrix defined from public program matrices must be of low rank. This implies that the determinant of all its minors of small size must be zero. The other equations are derived from the structure of the matrices used to define the low-rank matrix. The resulting amount of equations is not large enough to use Kipnis and Shamir's technique. Nevertheless, this minimum

number of equations implies a lower bound on the runtime of the attack, which we estimate. For an instance of size k , we have $\Omega(k^{28})$ runtime. Our concrete instance has size $k > 128$, which makes the attack practically impossible.

Details. Formally, for x **at infinity** define

$$g(x) = 2 \cdot |\text{constraints } x \text{ does not satisfy}| + |\text{nonzero constraints } x \text{ satisfies}|$$

and denote $g_i = g(\delta_i)$ where δ_i is an input assignment that has $x_j = \delta(i, j)$.

Trivially, we know that $g(x) \geq 1$ for all x . In fact, for a projectively safe system we can assume wlog that $g(x) \geq 3$. Indeed:

- Any δ_i input has at least the certifying constraint (j_i) , and if it is the only constraint it is involved in we can safely drop the variable i without changing the constraint system functionality.
- Any input which is not of the form $\text{const} \cdot \delta_i$ is involved in at least 2 certifying constraints.

Therefore, for each x we have at least two constraints, and at least one of them must not hold because of the projective safety condition (as x is at infinity).

Let $L_i^{(p,q)}$ consist of columns p and q of L_i and similarly $R_i^{(p,q)}$ consist of rows p and q of R_i . Let L_i^p stand for $L_i^{(p,p)}$ and R_i^p for $R_i^{(p,p)}$.

For any input x denote $D(x) \in \mathbb{F}^{2m \times 2m}$ the diagonal matrix

$$D(x) = \text{diag}([-\xi_0(x), \xi_0(x), \dots, -\xi_{m-1}(x), \xi_{m-1}(x)])$$

$P \in \mathbb{F}^{2m+1 \times 2m}$ the horizontal concatenation of $L_j^{(1,2)}$ for $0 \leq j < m$ and Q the vertical concatenation of $R_j^{(3,4)}$.

Now, we claim that

$$M(x) = PD(x)Q + \text{LowRank}_{\leq 2g(x)}$$

Indeed, for all i ,

$$M_i(x) = \begin{pmatrix} L_i^1 & L_i^2 & L_i^3 & L_i^4 \end{pmatrix} \cdot \begin{pmatrix} a_i(x)R_i^1 + c_i(x)R_i^2 - \xi(x)R_i^3 \\ d_i(x)R_i^1 + b_i(x)R_i^2 + \xi(x)R_i^4 \\ b_i(x)R_i^3 + c_i(x)R_i^4 \\ d_i(x)R_i^3 + a_i(x)R_i^4 \end{pmatrix}$$

so the product $PD(x)Q$ contains all the contributions of the top right 2×2 blocks of $U_i(x)$ from the **Algorithm 1**. It remains to add the contributions from the top left and bottom right blocks. If a constraint is nonzero but does hold, the contribution, which is equal to

$$\begin{pmatrix} L_i^1 & L_i^2 \end{pmatrix} \begin{pmatrix} a_i(x) & c_i(x) \\ d_i(x) & b_i(x) \end{pmatrix} \begin{pmatrix} R_i^1 \\ R_i^2 \end{pmatrix} + \begin{pmatrix} L_i^3 & L_i^4 \end{pmatrix} \begin{pmatrix} b_i(x) & c_i(x) \\ d_i(x) & a_i(x) \end{pmatrix} \begin{pmatrix} R_i^3 \\ R_i^4 \end{pmatrix}$$

has a rank 2 and if it does not hold it has a rank 4. Summing them together, we get a matrix with rank bounded by $2g(x)$.

Now, for a pair of inputs x, x' at infinity compute $\widetilde{M} = M(x')^{-1}M(x)$ and denote $\widetilde{D} = D(x')^{-1}D(x)$. Note that this can be computed from WE ciphertext as $\widetilde{M}(x) = M(x)$ for inputs at infinity.

Then, we claim that a matrix N defined via

$$Q\widetilde{M} = \widetilde{D}Q + N$$

has a low rank. Indeed, multiply both sides by $PD(x')$ on the left (which does not change the rank because it is a multiplication by a full rank $2m + 1 \times 2m$ matrix). We get:

$$PD(x')Q\widetilde{M} = PD(x')\widetilde{D}Q + N'$$

where $N' = PD(x')N$ has the same rank as N . This yields

$$(M(x') + \text{LowRank}_{\leq 2g(x')})M(x')^{-1}M(x) = M(x) + \text{LowRank}_{\leq 2g(x)} + N'$$

$$\text{LowRank}_{\leq 2g(x')} + \text{LowRank}_{\leq 2g(x)} = N'$$

The rank of N is at most $2g(x) + 2g(x')$. It may however be the case that there is a smaller upper bound on the rank of N if some additional relations hold on x and x' , but that upper bound is necessarily at least $2 \min(g(x), g(x'))$. Denote it $b - 1$. As $g(x) \geq 3$, it follows that $b \geq 7$.

As in [BIJ⁺20], the adversary can now set up a system of algebraic equations for the condition

$$\text{rank}(Q\widetilde{M} - \widetilde{Q}) < b,$$

treating the entries of Q and $\widetilde{Q} := \widetilde{D}Q$ as formal variables. Denote $k = 2m$. In the following calculation we will use the symbol $\approx$ to denote the leading term of a polynomial.

We obtain $\approx (\frac{k^b}{b!})^2$ degree b equations stating that determinant of each $b \times b$ minor of the matrix $Q\widetilde{M} - \widetilde{Q}$ vanishes. Additionally, there are equations stating that $\widetilde{Q}$ is a product of Q with some diagonal matrix (i.e., its columns are rescaled columns of Q). This is another $\approx \frac{k^3}{2}$ quadratic equations.

By itself, this amount of equations is not enough to launch a successful linearization attack - the total amount of degree b monomials in $2k^2$ variables is $\approx \frac{2^b}{(2b)!} k^{2b}$.

The amount of linear equations that we obtain is $\approx (\frac{1}{(b!)^2} + \frac{1}{2(2b-3)!})k^{2b}$. For $b \geq 7$ this value is significantly smaller than the amount of monomials. So, a naive attempt of launching this attack fails.

The authors of [BIJ⁺20] suggest that Kipnis and Shamir's relinearization technique [KS99] could potentially allow to launch the attack. While we were unable to do it, out of the abundance of caution we provide a trivial estimate that suggests that its runtime would be far beyond security level that we aim for. Indeed, for instance size k , even without any additional relinearization the size of the linear system is at least $O(k^{2b})$. As $b \geq 7$ and the runtime for solving a linear system with r equations is at least $\Omega(r^2)$, we have $\Omega(k^{28})$ runtime. Our concrete instance has size $k > 128$, which makes the attack practically impossible.

As a theoretical mitigation, we suggest enforcing that a constraint system has superconstant minimal value of $g(x) > m^\varepsilon$. This can be achieved by repeating each gate m^ε times.

Overall, we consider evidence for the linearization attack inconclusive and believe that further research is required in this direction.

3.5 Security claims

Assumption 1. (Theoretical security claim) for a projectively safe constraint system with $n > O(\lambda)$, $\varepsilon := \varepsilon(\lambda) < 1$, and $g(x) > O(m^\varepsilon)$ the pair (Algorithm 2, Algorithm 3) is an extractable witness encryption scheme.

Remark. Extractability is, of course, non-falsifiable. We prefer it to the weaker notion of soundness because the constraint systems that we compose our WE scheme with are SNARK verifiers that always have a valid (even if computationally infeasible to find) solutions.

Assumption 2. (Concrete security claim) the circuits in the next chapter, combined with the instantiation of our WE scheme provide at least 100 bits of security.

Remark. We could provide higher security level if not for the curve choice - BN254 [BN06], for which we instantiate our circuits is known to have around 100 bits of security for the composition of our scheme and a SNARK.

4 Projectively Safe Gadgets

In this section we develop a collection of arithmetic gadgets for building *projectively safe* constraint systems, as required by the AADP framework. Projective safety severely restricts the form of admissible constraints and, in particular, rules out direct use of standard arithmetic operations such as multiplication or conditional branching. Our goal is to show that, despite these restrictions, it is still possible to express the computations needed to verify pairing-based SNARK proofs.

4.1 Notation

In order to build a set of gadgets we need to keep track of projective safety of the resulting constraint system.

Definition 5. Recall that in the terminology of [Definition 3](#) for (not necessarily projectively safe) constraint system we call a constraint indexed by (j_i) of form

$$(j_i) : x_i^2 = a(x_0, \dots, x_{i-1}) \cdot b(x_0, \dots, x_n)$$

a **certificate** for variable x_i .

If a variable x_i has a **certificate** in the constraint system we say that this variable is **certified**.

Note 2. A constraint system is projectively safe iff all of its variables are certified.

To apply a gadget correctly, it is necessary to specify both an ordering of the variables appearing in the gadget and the certification status of these variables. In particular, some variables are required to be certified prior to applying the gadget, some will be certified by the gadget itself, and others may remain uncertified.

To track this information, we annotate each gadget with an ordered list of its variables. Each variable in this list is marked according to its role:

- $\underline{\underline{x}}$: the variable x is required to be certified before applying the gadget.
- $\underline{x}$: the variable x is certified by the gadget.
- x : the variable x is not certified by the gadget.

Note 3. When a new gadget is allocated, its constraints (under pre-certification conditions) must certify intermediate variables - which is true for all our gadgets. Outputs of the gadget are not always certified, so this information needs to be tracked explicitly; it is quite easy to do because all our gadgets except for the Poseidon S-box certify their output, so we never obtain non-certified variables. The gadget for the Poseidon S-box is unique, because it certifies its inputs, and keeps the output uncertified, so when rounds are composed, the resulting composition certifies its intermediate variables. The global outputs of the Poseidon hash are certified explicitly via bitcheck, as written explicitly in the final gadget ([Gadget 15](#)).

In the description of gadgets, we will use the pseudocode notation resembling the Circom language [\[BIM⁺23\]](#): we will use the symbol " $=$ " to denote constraints, the symbol " $\leftarrow$ " to denote the hint to the constraint system solver and the symbol " $\Leftarrow$ " to denote both at the same time. For readability, we write $\mathbb{1} = x_0$ (as $x_0 = 1$ in the affine solution).

Note 4. Sometimes, gadgets will have separate **completeness** and **soundness** properties. Soundness properties are the conditions that are enforced by constraints, i.e. they happen even if the constraint system solver does not comply with our hints (denoted by " $\leftarrow$ "). Completeness properties are the statements of the form "if solver acts in a recommended way, the system is solvable (and some additional conditions hold)". In the cases where these properties meaningfully differ, we will write them separately.

4.2 First examples

A central limitation of projectively safe constraint systems is that we do not know how to directly multiply field elements without incurring a logarithmic overhead in additional certification constraints. Indeed, if one tries to naively constrain $\mathbb{1} \cdot c = a \cdot b$, the variable c now needs to be certified. There are several ways of certifying an element, discussed in [Section 4.5](#), but all of them incur $O(\log(|\mathbb{F}|))$ additional witness and constraint cost.

Our budget is extremely tight - for example, 20.000 witness, 20.000 constraint system yield ciphertext sizes ≈ 1 PB. A single scalar multiplication of a point of elliptic curve by a 256-bit scalar requires ≈ 1500 multiplications - which by itself would already yield ciphertext sizes of several exabytes!

Nevertheless, it is possible to perform useful computations in a projectively safe manner by using, instead of multiplication, a gadget that computes a square root of a product.

Recall that we work over a base field $\mathbb{F}$, which is a field of coefficients, and all variables can take values in algebraic closure $\overline{\mathbb{F}}$. Some of our gadgets will operate in a certain extension $\mathcal{K}/\mathbb{F}$ and an odd-order subgroup $G_{\text{odd}} \subset \mathcal{K}^*$.

Gadget 1: Multiply and square root

Functionality:

Given certified variables $a, b \in \overline{\mathbb{F}}$, output a variable $c \in \overline{\mathbb{F}}$ such that $c^2 = ab$, i.e., c is a square root of ab .

Variable ordering:

$\underline{a}, \underline{b} \prec \underline{c}$

Builder:

$$\begin{aligned} c^2 &= a \cdot b \\ c &\leftarrow \sqrt{ab} \end{aligned}$$

We write $c \leftarrow \sqrt{ab}$ as a shorthand for application of [Gadget 1](#): i.e. introducing a variable c , enforcing the constraint $c^2 = a \cdot b$, and assigning c a square root of ab as a witness; and specific choice of the square root will be inferred from the context.

Remark 2. The output variable c is certified by the squaring constraint $c^2 = a \cdot b$, while the input variables a and b must be certified prior to applying the gadget.

Our most important gadgets (exponentiations, described in this subsection and Miller loop, described in [Section 4.6](#)) will use this square root gate as a basic block.

Because the computation of the square root is not unique, we will use them in the odd multiplicative subgroup $G_{\text{odd}} \subset \mathcal{K}^*$, where the square root always exists and unique. Adversary could, of course, choose the roots not from the subgroup - but any multiplicative factor that it introduces belongs to the group of power-of-2 roots of unity μ_{2^∞} .

There is an embedding $G_{\text{odd}} \hookrightarrow \overline{\mathbb{F}}^* / \mu_{2^\infty}$. This means that while an adversary could introduce some μ_{2^∞} -twist in the computation result, this never allows it to incorrectly pass an equality check.

This situation leads to the divergence of completeness and soundness properties: from the perspective of the constraint system, solver is only constrained up to the μ_{2^∞} -twist and from the completeness perspective solver is *advised* to keep the values in G_{odd} so that it can solve the system correctly.

Gadget 2: OddGroupFixpowExp - Odd-order subgroup exponent with fixed power

Functionality:

Given a certified variable $x \in \overline{\mathbb{F}}$ and a fixed exponent $N \in \mathbb{N}$, let $\ell = \lfloor \log_2 N \rfloor + 1$.

The gadget outputs a variable y such that

(Soundness:) any assignment satisfying the constraints obeys

$$y^{2^\ell} = x^N$$

(Completeness:) moreover, if $x \in G_{\text{odd}} \subset \mathcal{K}^*$, then solver can choose to have $y = x^M$ where $M = N/2^\ell \pmod{|G_{\text{odd}}|}$.

Variable ordering:

$$\underline{x} \prec \underline{y[1]} \prec \underline{y[2]} \dots \prec \underline{y[\ell]} = \underline{y}$$

Builder:

Let $N = \sum_{i=0}^{\ell-1} 2^i \cdot \text{bits}[i]$ be a bit decomposition of N .

```

 $y[0] \Leftarrow 1$ 
for  $0 \leq i \leq \ell - 1$  do
  if  $\text{bits}[i] = 0$  then
     $y[i+1] \Leftarrow \sqrt{y[i] \cdot 1}$ 
  else
     $y[i+1] \Leftarrow \sqrt{x \cdot y[i]}$ 
  end if
end for
return  $y \Leftarrow y[\ell]$ 

```

Lemma 7 (Soundness properties). Let $N = \sum_{i=0}^{\ell-1} 2^i \cdot \text{bits}[i]$ and let y be the output of **Gadget 2**. Then any assignment satisfies $y^{2^\ell} = x^N$.

Proof. We track the exponent of x throughout the construction. Let's define coefficient e_i as follows:

$$e_i = \sum_{s=0}^{i-1} \text{bits}[s] \cdot 2^s.$$

We will prove by induction that $y[i]^{2^i} = x^{e_i}$. The base case $y[0] = 1$ is clear. For the inductive step, observe

$$y[i+1]^2 = \begin{cases} y[i], & \text{bits}[i] = 0, \\ x \cdot y[i], & \text{bits}[i] = 1, \end{cases}$$

Computing 2^i -th power of both left and right hand side and using the inductive assumption we get

$$y[i+1]^{2^{i+1}} = \begin{cases} x^{e_i}, & \text{bits}[i] = 0, \\ x^{2^i + e_i}, & \text{bits}[i] = 1, \end{cases}$$

which yields $y^{2^{i+1}} = x^{e_{i+1}}$ because $e_{i+1} = e_i + 2^i \cdot \text{bits}[i]$. □

Lemma 8. Assume $x \in G_{\text{odd}}$, and let y be the output of **Gadget 2**. Then there exists a satisfying assignment to all intermediate variables such that the output equals $y = x^M$, where $M \equiv N/2^\ell \pmod{|G_{\text{odd}}|}$.

Proof. Order of G_{odd} is odd, so the squaring map $g \mapsto g^2$ is a bijection on G_{odd} , and every element of G_{odd} admits a (unique) square root within the subgroup.

Starting from $y[0] = 1$, we choose at each step $y[i+1] \in G_{\text{odd}}$ to be the square root in G_{odd} of $y[i]$ or $x \cdot y[i]$, depending on $\text{bits}[i]$. This satisfies all constraints of the gadget. By the previous lemma, we obtain a unique $y \in G_{\text{odd}}$ that satisfies $y^{2^\ell} = x^N$, which is x^M . $\square$

Gadget 3: OddGroupVarpowExp - Odd-order subgroup exponent with variable power
Functionality:

Notation and functionality is almost the same as functionality of **Gadget 2** except N is now not a constant, but a bitstring of variables.

Given a certified variable $x \in \mathbb{F}$ and a bit-vector of variables $\text{bits}[0], \dots, \text{bits}[\ell-1] \in \{0, 1\}$ encoding $N = \sum_{i=0}^{\ell-1} 2^i \cdot \text{bits}[i]$, the gadget outputs a variable y such that

(Soundness:) any assignment satisfying the constraints obeys

$$y^{2^{\ell+1}} = x^N$$

(Completeness:) moreover, if $x \in G_{\text{odd}}$, then there exists a satisfying assignment with $y = x^M$ where $M = N/2^{\ell+1}$.

Variable ordering:

$$\underline{x}, \underline{\text{bits}}[] \prec \underline{s}[0] \prec \underline{y}[1] \prec \underline{s}[1] \prec \underline{y}[2] \dots \prec \underline{s}[\ell-1] \prec \underline{y}[\ell] = \underline{y}$$

Builder:

```

 $y[0] \Leftarrow 1$ 
for  $0 \leq i \leq \ell - 1$  do
   $s[i] \Leftarrow \sqrt{\text{bits}[i] \cdot x}$ 
   $y[i+1] \Leftarrow \sqrt{(1 - \text{bits}[i] + s[i]) \cdot y[i]}$ 
end for
return  $y \Leftarrow y[\ell]$ 

```

Lemma 9. The output of **Gadget 3** satisfies

$$y^{2^{\ell+1}} = x^N.$$

Proof. The proof almost verbatim follows the proof of **Lemma 7**. As before, define

$$e_i = \sum_{s=0}^{i-1} \text{bits}[s] \cdot 2^s.$$

Now, we prove by induction that $y[i]^{2^{i+1}} = x^{e_i}$. The base case is still $y[0] = 1$. The difference in the inductive step is that now we have:

$$s[i]^2 = \text{bits}[i] \cdot x$$

$$y[i+1]^2 = (1 - \text{bits}[i] + s[i]) \cdot y[i]$$

Observe that $s[i] = 0$ if $\text{bits}[i] = 0$ and $s[i] = \sqrt{x}$ if $\text{bits}[i] = 1$. The choice of the square root for different i -s is not necessarily the same.

Denote $c_i = 1 - \text{bits}[i] + s[i]$. We can see that

$$c_i^2 = \begin{cases} 1, & \text{bits}[i] = 0, \\ x, & \text{bits}[i] = 1, \end{cases}$$

Now, taking the constraint $y[i+1]^2 = c_i \cdot y[i]$ to the power 2^{i+1} we obtain

$$y[i+1]^{2^{i+2}} = \begin{cases} y[i]^{2^{i+1}}, & \text{bits}[i] = 0, \\ x^{2^i} y[i]^{2^{i+1}}, & \text{bits}[i] = 1, \end{cases}$$

which by inductive assumption yields the inductive step:

$$y[i+1]^{2^{i+2}} = x^{e_i + 2^i \text{bits}[i]} = x^{e_{i+1}}$$

□

Lemma 10 (Completeness properties). Assume $x \in G_{\text{odd}}$, and let y be the output of [Gadget 3](#). Then there exists a satisfying assignment to all intermediate variables such that the output equals $y = x^M$, where $M \equiv N/2^{\ell+1} \pmod{|G_{\text{odd}}|}$.

The claim follows exactly as in [Lemma 8](#).

4.3 A Proof System for AADPs

In this section we present a proof system whose verification procedure can be embedded into AADPs. The central difficulty is that verification must be expressible using *projectively safe* arithmetic constraints.

Notation.

- $\mathcal{G}_1$, $\mathcal{G}_2$, and $\mathcal{G}_t$ denote cyclic groups equipped with a non-degenerate bilinear pairing

$$\langle \cdot, \cdot \rangle : \mathcal{G}_1 \times \mathcal{G}_2 \rightarrow \mathcal{G}_t$$

arising from a Type-3 pairing-friendly elliptic curve. In our concrete estimates, we instantiate these groups using the BN254 curve [\[BN06\]](#).

- We fix generators $[1]_1 \in \mathcal{G}_1$ and $[1]_2 \in \mathcal{G}_2$, and write $[1]_t := \langle [1]_1, [1]_2 \rangle \in \mathcal{G}_t$. For $\star \in \{1, 2, t\}$ and x in the appropriate scalar field, we use the notation $[x]_\star := x \cdot [1]_\star$.
- The base field of $\mathcal{G}_1$ is the field $\mathbb{F}$, which also serves as the underlying field for our WE construction.
- We denote by $\mathcal{K} = \mathbb{F}^d$ the degree- d extension field used for embedding $\mathcal{G}_2$ and $\mathcal{G}_t$.
- All groups $\mathcal{G}_1$, $\mathcal{G}_2$, and $\mathcal{G}_t$ have prime order r , and the corresponding scalar field is denoted by $\mathbb{F}_r$.

Design goals. AADP-friendly SNARK verification should satisfy the following structural conditions:

- Verification must be pairing-based.
- Pairings may involve only fixed elements of $\mathcal{G}_2$. As we will see later, the computation of the Miller loop is possible in this case.
- All arithmetic outside pairings, Miller loops, and exponentiation must be kept to a minimum, as general field operations are extremely costly when encoded as projectively safe constraints. In particular, elliptic curve operations in $\mathcal{G}_1$ should also be avoided.
- Small amount of hashing is acceptable, as we are able to effectively implement a Poseidon hash [\[GKR⁺19\]](#).

Our design is inspired by Garuda [DMS24], combining KZG-style polynomial commitments [KZG10] with a circuit-specific trusted setup.

Quadratic Arithmetic Programs. We briefly recall the notion of QAPs, which we will later modify to better suit our setting.

Fix the following parameters:

- n : the number of witness variables;
- $\text{pub} < n$: the number of public input variables;
- m : the number of constraints;
- $\mathcal{Z} \subset \mathbb{F}_r$: a constraint set, subject to $|\mathcal{Z}| = m$

We write $\mathcal{Z} = \{z_0, \dots, z_{m-1}\}$ and define the vanishing polynomial $Z(x) := \prod_{i=0}^{m-1} (x - z_i)$, which has degree m and vanishes exactly on the points in $\mathcal{Z}$.

Definition 6 (QAP). *QAP instance with these parameters is given by a set of triples of polynomials $(A^i(x), B^i(x), C^i(x))$ for $0 \leq i < n$ of degree $< m$ and a public input vector $x = (x_0, \dots, x_{\text{pub}-1})$.*

QAP witness is given by a pair of vectors (w, h) where $w = (w_0, \dots, w_{n-1})$, $h = (h_0, \dots, h_{m-2})$ such that $w_i = x_i$ for $i < \text{pub}$ and

$$A(x)B(x) - C(x) = Z(x)H(x)$$

where

$$\begin{aligned} A(x) &= \sum_{i=0}^{n-1} w_i A^i(x) & B(x) &= \sum_{i=0}^{n-1} w_i B^i(x) \\ C(x) &= \sum_{i=0}^{n-1} w_i C^i(x) & H(x) &= \sum_{i=0}^{m-2} h_i x^i \end{aligned}$$

Consider some rank one constraint system (R1CS)

$$(j) : \left(\sum_{i=0}^n a_j^i w_i \right) \cdot \left(\sum_{i=0}^n b_j^i w_i \right) = \left(\sum_{i=0}^n c_j^i w_i \right)$$

with public input condition $w_i = x_i$ for $i < \text{pub}$.

It is a standard fact that this system of constraints is equivalent to the QAP defined via interpolation:

$$A^i(z_j) = a_j^i \quad B^i(z_j) = b_j^i \quad C^i(z_j) = c_j^i$$

Indeed, for a vector w a polynomial $A(x)B(x) - C(x)$ is divisible by $Z(x)$ if and only if the original R1CS constraint system was satisfied.

We modify the standard QAP formulation in two respects.

First, we restrict to the case $A = B$. QAPs of this form are commonly referred to as *squaring programs*, as they directly verify only constraints of the form $a^2 = c$. Since general multiplication can be expressed using addition and squaring (i.e., $ab = \frac{(a+b)^2 - a^2 - b^2}{2}$ over $\mathbb{F}$ where $\text{char}(\mathbb{F}) \neq 2$), this restriction does not significantly reduce the expressiveness of the resulting SNARK.

Second, we alter the handling of public inputs. Rather than binding a designated subset of witness variables to public inputs, we inject public inputs directly into the QAP equation as fixed coefficients on the right-hand side. In what follows, we restrict attention to a single public input, denoted by u .

Definition 7. *Modified squaring QAP is the same as the normal QAP with the following changes: $B^i(x) = A^i(x)$ for all $0 \leq i < n$, there is no direct condition on public inputs and instead the final equation is*

$$A^2(x) - C(x) = Z(x)H(x) + u$$

where $u \in \mathbb{F}_r$ is a fixed public input shared across all constraints.

This naturally corresponds to the following modified squaring-R1CS notion:

Definition 8. A modified squaring R1CS consists of constraints indexed by j , each of the form

$$(j) : \left(\sum_{i=0}^{n-1} a_j^i w_i \right)^2 - \left(\sum_{i=0}^{n-1} c_j^i w_i \right) = u$$

where $w_i \in \mathbb{F}_r$ are witness variables and $u \in \mathbb{F}_r$ is a fixed public input shared across all constraints.

Modified squaring R1CS is equivalent to modified squaring QAP in exactly the same way that a standard R1CS is equivalent to a standard QAP, namely via polynomial interpolation of constraint coefficients at the evaluation points $\{z_0, \dots, z_{m-1}\}$ and the usual vanishing-polynomial argument.

We now explain how a *regular squaring R1CS* with constraints of the form

$$\left(\sum_{i=0}^{n-1} a_j^i w_i \right)^2 - \left(\sum_{i=0}^{n-1} c_j^i w_i \right) = 0$$

can be simulated using modified squaring R1CS, in which every constraint is required to equal a fixed public input u .

To do so, we introduce an auxiliary witness variable w_0 and add the constraint

$$(0)' : w_0 = u.$$

This enforces that w_0 is bound to the public input while remaining a witness variable. Each original constraint indexed by j is then replaced by

$$(j+1)' : \left(\sum_{i=0}^{n-1} a_j^i w_i \right)^2 - \left(\sum_{i=0}^{n-1} c_j^i w_i \right) + w_0 = u.$$

Substituting $w_0 = u$ into the above equation yields

$$\left(\sum_{i=0}^{n-1} a_j^i w_i \right)^2 - \left(\sum_{i=0}^{n-1} c_j^i w_i \right) = 0,$$

which is exactly the original squaring R1CS constraint. Thus, every solution to the modified squaring R1CS corresponds to a solution of the original system, and vice versa.

Remark 3. Normally we desire to have at least **two** public inputs - an actual public input and the value 1. However, as long as we verify $u = 1 \pmod{2}$ outside of the circuit, the value 1 can be recovered in the circuit by bit-splitting u :

$$u = \sum_i 2^i b_i; \text{ now } b_0 \text{ can be used as } 1$$

While this is definitely non-standard, our verification budget is so tight that this optimization is justified.

Informal Description of Our Proof System We plan to use a batched version of KZG commitment scheme: for a collection of polynomials $P_1, \dots, P_k$ we define

$$\mathcal{P} = \text{VecKZGCommit}(P_1, \dots, P_k) := \left[\sum \alpha_i P_i(\tau) \right]_1$$

The vector opening for $P_i(z) = v_i$ is similar to the standard version

$$\langle \mathcal{P}, [1]_2 \rangle = \langle \mathcal{Q}, [\tau]_2 - z \cdot [1]_2 \rangle + \sum v_i [\alpha_i]_t$$

where $\mathcal{Q}$ is the commitment to the quotient polynomial.

We intend to provide a commitment to A, C, H , that will be denoted $\mathcal{P}$. It will be twisted by another setup element γ and CRS will only contain divisible by γ elements $\mathcal{H}^i = [\gamma \alpha_H \tau^i]_1$ and $\mathcal{W}^i = [\gamma (\alpha_A A^i(\tau) +$

$\alpha_C C^i(\tau)]_1$. This will enforce that $\mathcal{P}$ is a commitment to polynomials A, C, H satisfying linear constraints - i.e. that indeed $A = \sum w_i A^i(x)$, $C = \sum w_i C^i(x)$.

The opening argument now takes the form

$$\langle \mathcal{P}, [\gamma^{-1}]_2 \rangle = \langle \mathcal{Q}, [\tau]_2 - z \cdot [1]_2 \rangle + \sum v_i [\alpha_i]_t$$

To check modified squaring QAP (Definition 7) we also need to compute $Z(z)$. We include this evaluation in the pairing check in a following way - the quotient term $(Z(x) - v_Z)/(x - z)$ is added into $\mathcal{Q}$, and the contribution of Z in the left hand side is directly added as a constant. The check takes the following form:

$$\langle \mathcal{P}, [\gamma^{-1}]_2 \rangle + [\alpha_Z Z(\tau)]_t = \langle \mathcal{Q}, [\tau]_2 - z \cdot [1]_2 \rangle + \sum_{U \in (A, B, H, Z)} v_U [\alpha_U]_t$$

Furthermore, addition in $\mathcal{G}_2$ is expensive, so we open brackets to obtain the final form of opening argument

$$\langle \mathcal{P}, [\gamma^{-1}]_2 \rangle + [\alpha_Z Z(\tau)]_t = \langle \mathcal{Q}, [\tau]_2 \rangle - z \cdot \langle \mathcal{Q}, [1]_2 \rangle + \sum_{U \in (A, C, H, Z)} v_U [\alpha_U]_t$$

Formal Description of Our Proof System Our SNARK is a tuple of PPT algorithms (Setup, Prove, Verify), where the algorithms are defined as follows:

- **Setup**(λ, pp) $\rightarrow$ CRS: On input a (unary encoded) security parameter λ and public parameters pp defining a modified squaring QAP instance, it proceeds as in Algorithm 4 and outputs common reference string CRS. In addition to what was discussed in the informal description setup also adds $[\alpha_U \tau^i]_1$ for all $U \in (A, C, H, Z)$, these will be required by prover to construct commitment $\mathcal{Q}$ to quotient polynomial.

Algorithm 4 Algorithm SNARK.Setup

$\tau \leftarrow \mathbb{F}_r$ $\gamma \leftarrow \mathbb{F}_r$ $\alpha_U \leftarrow \mathbb{F}_r$ for $U \in (A, C, H, Z)$ $\text{CRS} \leftarrow ()$ for $0 \leq i < n$ do $\mathcal{W}^i \leftarrow [\gamma(\alpha_A A^i(\tau) + \alpha_C C^i(\tau))]_1$ $\text{CRS} \leftarrow \text{CRS} (\mathcal{W}^i)$ end for for $0 \leq i < m - 1$ do $\mathcal{H}^i \leftarrow [\gamma \alpha_H \tau^i]_1$ $\text{CRS} \leftarrow \text{CRS} (\mathcal{H}^i)$ end for	for $U \in \{A, C, H, Z\}$ do if $U \in \{A, C\}$ do $k \leftarrow m - 1$ if $U \in \{H\}$ do $k \leftarrow m - 2$ if $U \in \{Z\}$ do $k \leftarrow m$ for $0 \leq i < k$ do $\mathcal{T}_U^i \leftarrow [\alpha_U \tau^i]_1$ $\text{CRS} \leftarrow \text{CRS} (\mathcal{T}_U^i)$ end for end for $\text{CRS} \leftarrow \text{CRS} ([1]_2, [\tau]_2, [\gamma^{-1}]_2)$ $\text{CRS} \leftarrow \text{CRS} ([\alpha_Z Z(\tau)]_t, [\alpha_U]_t \text{ for } U \in \{A, C, H, Z\})$ return CRS
--	---

- **Prove**(CRS, (w, h)) $\rightarrow$ Proof: On input CRS and the solution (w, h) to circuit defined by pp with a public input u , it proceeds as in Algorithm 5 and outputs a proof Proof. In short, using $\mathcal{W}^i, \mathcal{H}^i$ from the CRS, the prover forms a $\mathcal{G}_1$ commitment $\mathcal{P}$ to A, C, H . Then derives a Fiat Shamir challenge z from P and the public input pp . The prover evaluates the QAP polynomials at z and constructs KZG-style quotient commitments for $(U(x) - U(z))/(x - z)$, for each $U \in \{A, C, H, Z\}$, which are aggregated into a single $\mathcal{G}_1$ element $\mathcal{Q}$. The final proof consists of $(P, \mathcal{Q}, u, \{U(z)\})$.

Algorithm 5 Algorithm SNARK.Prove

Input: Solution (w, h) to a modified squaring QAP instance with a public input u .
 $\mathcal{P} \leftarrow \sum w_i \mathcal{W}^i + \sum h_i \mathcal{H}^i$
 $z \leftarrow \text{Hash}(\text{pp}, \mathcal{P}, u) \in \mathbb{F}_r$
 $v_U \leftarrow U(z)$ for $U \in \{A, C, H, Z\}$
 $Q_U(x) \leftarrow (U(x) - U(z))/(x - z)$ for $U \in \{A, C, H, Z\}$
 $\mathcal{Q} \leftarrow \sum (Q_U)_{(i)} \mathcal{T}_U^i$
return Proof = $(u, \mathcal{P}, \mathcal{Q}, v_U \text{ for } U \in \{A, C, H, Z\})$

- **Verify**(CRS, Proof) $\rightarrow 0/1$: On input CRS and a proof Proof, it verifies a proof by checking that the committed polynomials satisfy the modified squaring QAP identity at the challenge point z that it obtains as a hash of public parameters, commitment $\mathcal{P}$ and public input u . The verifier checks the arithmetic consistency condition $v_A^2 - v_C = v_H \cdot v_Z + u$, which ensures that the claimed evaluations correspond to a valid solution of the modified squaring constraints. Then, the verifier runs a single structured pairing check that is the vectorized opening argument for A, C, H, Z , discussed in the informal overview. Additionally, verifier checks $u = 1 \pmod{2}$ to enforce the public input trick (Remark 3).

Algorithm 6 Algorithm SNARK.Verify

Input: Proof = $(u, \mathcal{P}, \mathcal{Q}, v_U \text{ for } U \in \{A, C, H, Z\})$
 $z \leftarrow \text{Hash}(\text{pp}, \mathcal{P}, u)$
Require: $u = 1 \pmod{2}$
Require: $v_A^2 - v_C = v_H \cdot v_Z + u$
Require $\langle \mathcal{P}, [\gamma^{-1}]_2 \rangle + [\alpha_Z Z(\tau)]_t = \langle \mathcal{Q}, [\tau]_2 \rangle - z \cdot \langle \mathcal{Q}, [1]_2 \rangle + \sum_U v_U [\alpha_U]_t$

Theorem 2 (Correctness). Given CRS generated by the algorithm Setup for a modified squaring QAP instance, and (w, h) be a satisfying witness for public input u , i.e., $A(x)^2 - C(x) = Z(x)H(x) + u$, then a proof generated by the algorithm Prove is always accepted by the algorithm Verify.

Proof. Let $z := \text{Hash}(\text{pp}, \mathcal{P}, u)$ be the challenge and $v_U := U(z)$ for $U \in \{A, C, H, Z\}$. Evaluating the QAP identity at z yields $A(z)^2 - C(z) = Z(z)H(z) + u$, which is exactly the verifier's arithmetic condition $v_A^2 - v_C = v_H v_Z + u$.

By construction of the CRS and prover commitments,

$$\begin{aligned} \langle \mathcal{P}, [\gamma^{-1}]_2 \rangle + [\alpha_Z Z(\tau)]_t &= [\alpha_A A(\tau) + \alpha_C C(\tau) + \alpha_H H(\tau) + \alpha_Z Z(\tau)]_t \\ &= \left[\sum_{U \in \{A, C, H, Z\}} \alpha_U U(\tau) \right]_t. \end{aligned}$$

For each $U \in \{A, C, H, Z\}$, the prover commits to $Q_U(x) = (U(x) - U(z))/(x - z)$ and aggregates these into $\mathcal{Q}$, which implies $\langle \mathcal{Q}, [\tau]_2 \rangle - z \cdot \langle \mathcal{Q}, [1]_2 \rangle = \left[\sum_U \alpha_U (U(\tau) - U(z)) \right]_t$. Adding $\sum_U v_U [\alpha_U]_t$ and using $v_U = U(z)$ yields $\left[\sum_U \alpha_U U(\tau) \right]_t$, which matches the expression above. Hence, the verifier's pairing equation holds. $\square$

Theorem 3 (Knowledge soundness). This SNARK is knowledge sound in the generic group model (and ROM for Fiat-Shamir transform).

Proof. Recall that generic group model (GGM) is a strong idealized model that assumes that adversary is only able to output proof elements that are linear combinations of the elements of CRS. Moreover, adversary is not allowed to inspect CRS elements themselves, only manipulating them in a black-box fashion. It means that adversary's strategy is not dependent on the setup and still must succeed against non-negligible fraction of setups.

Step 1. $\mathcal{P}$ only contains the terms from the CRS (defined in [Algorithm 4](#)) divisible by γ . Otherwise, the pairing term $\langle \mathcal{P}, [\gamma^{-1}]_2 \rangle$ will contain a nontrivial monomial of degree -1 in γ that does not cancel out by anything in the pairing equation. So wlog $\mathcal{P} = \sum w_i \mathcal{W}^i + \sum h_i \mathcal{H}^i$.

Step 2. A random challenge z is produced based on a commitment to $\mathcal{P}$ and public input u . As long as claimed evaluations v_U for $U \in \{A, C, H, Z\}$ are correct and $v_A^2 - v_C = v_H v_Z + u$ holds, the corresponding QAP equation holds by Schwartz-Zippel lemma. It remains to show that the opening argument is correct.

Step 3. $\mathcal{Q}$ can only contain terms from $\mathcal{G}_1$ that correspond to monomials not divisible by γ for the reason analogous to Step 1. Therefore, it has the form $\mathcal{Q} = \sum (q_U)_i \mathcal{T}_U^i$. Denote $Q_U(x) = (q_U)_i x^i$.

Step 4. Consider all the terms divisible by α_U in the equation for each $U \in \{A, C, H, Z\}$. For each U , the check implies $U(\tau) - v_Z = (\tau - z) \cdot Q_U(\tau)$, which implies $v_U = U(z)$.

Step 5. We therefore extracted from an adversary the values w_i, h_i representing a correct modified squaring QAP witness.

□

Embedding strategy The rest of this chapter is devoted to the embedding of the verifier [Algorithm 6](#) into AADP-based witness encryption scheme. Roughly, we will need the following gadgets:

- Non-native arithmetic in the field $\mathbb{F}_r$.
- Poseidon hash.
- Extension field constructor. Our circuits will have coefficients over $\mathbb{F}$ and we will construct the basis of the extension $\mathcal{K} = \mathbb{F}^d$ inside of the circuit.
- Miller loop with a fixed $\mathcal{G}_2$ point.
- Final exponent (using the method already described in [Section 4.2](#).)

4.4 Finding the extension basis

Given a projectively safe constraint system over $\mathbb{F}$, we would like to use the elements of the extension $\mathcal{K} = \mathbb{F}^d$. The gadget described in this section constructs a basis of $\mathcal{K}/\mathbb{F}$.

Let $E(x)$ be a degree d monic polynomial defining the extension: $\mathcal{K} \simeq \mathbb{F}[x]/(E(x))$.

A straightforward idea is to construct a (projectively safe) constraint system that validates, for a certain element s that $E(s) = 0$ and then to use the powers of s as the basis of $\mathcal{K}$.

Gadget 4: FindExtensionBasis

Functionality:

Constrain and output variables $s_1, \dots, s_{d-1}$ satisfying $\forall i : s_i = s^i, E(s) = 0$.

Variable ordering:

No ordering dependency. All variables $\underline{s_1}, \dots, \underline{s_{d-1}}$ are certified.

Builder:

$s_0 \leftarrow \mathbb{1}$

```

 $s_1 \leftarrow \text{any root of } E(x) \text{ in } \overline{\mathbb{F}}$ 
for  $2 \leq i < d$  do
   $s_i \leftarrow s_1 \cdot s_{i-1}$ 
   $\mathbb{1} \cdot s_i = s_1 \cdot s_{i-1}$ 
   $\mathbb{1} \cdot \left(-\sum_{i=0}^{d-1} E_{(i)} s_i\right) = s_1 \cdot s_{d-1}$ 
end for
 $\triangleright$  By this point, all  $s_i$ -s are constrained correctly, but nothing is certified  $\triangleleft$ 
for  $0 \leq i < d$  do
   $R(x) \leftarrow x^{2^i} \bmod E(x)$ 
   $s_i^2 = \mathbb{1} \cdot \sum_{i=0}^{d-1} R_{(i)} s_i$ 
   $\triangleright$  These constraints hold automatically given the previous ones and certify individual  $s_i$ -s  $\triangleleft$ 
end for
return  $(s_1, \dots, s_{d-1})$ 

```

Lemma 11. Variables $s_1, \dots, s_{d-1}$ satisfying **Gadget 4** satisfy $s_i = s^i$ and $E(s) = 0$. **Gadget 4** is projectively safe.

Proof. The constraints from the first set guarantee that $s_i = s^i$ by induction and the last constraint reduces to $E(s) = 0$. The second set of constraints always holds given the first one and makes the constraint system projectively safe. $\square$

4.5 Certification toolbox

Now we will discuss some tricks that allow for variable certification.

Gadget 5: Certification by binary decomposition

Notes:

Let $\ell = \lfloor \log q \rfloor$

Functionality:

Certify variable a and check that $a \in \mathbb{F}$

Variable ordering:

There is no ordering on variables $\underline{b}_0, \dots, \underline{b}_\ell$

Builder:

```

 $b[0 \dots \ell] \leftarrow \text{bits}(a)$ 
for  $0 \leq i \leq \ell$  do
   $b_i^2 = \mathbb{1} \cdot b_i$ 
end for

```

Note 5. One can see that variable a does not appear in the constraints or variable ordering. Instead, we alias it to the linear combination $\sum_{i=0}^{\ell} 2^i b_i$. So, formally, there is no such variable as a . When any gadget's ordering would include variable a we will replace it with all b_i -s.

An alternative way to certify uses the exponentiation. This version of exponentiation is different from **Gadget 2**: it doesn't need to work in G_{odd} , but its certification properties are reversed: it takes uncertified

input, certifies it, but leaves the output uncertified. Another difference is that **Gadget 5** is defunct for the zero input.

Gadget 6: Alternative exponent with fixed power

Functionality:

Given variable x and a constant power $t > 2$ output x^t if $x \neq 0$. For zero input it is allowed to output anything.

Variable ordering:

$\underline{x} \prec y[0] \prec \dots \prec y[N-1] \prec y[N]$

Builder:

Find constants $\text{bits}[0 \dots (N-1)]$ such that $t = 1 + 2^N + \sum_{i=0}^{N-1} \text{bits}[i]2^i$

```

 $x^2 = 1 \cdot y[0]$ 
for  $0 \leq i < N-1$  do
  if  $\text{bits}[N-1-i] = 1$  then
     $y[i]^2 = 1 \cdot y[i+1]$ 
     $y[i+1] \leftarrow y[i]^2$ 
  else
     $y[i]^2 = x \cdot y[i+1]$ 
     $y[i+1] \leftarrow \frac{y[i]^2}{x}$ 
  end if
end for
return  $y[N]$ 

```

Lemma 12. For **Gadget 6**, the following holds: If $x \in \mathbb{F}$ and $x \neq 0$, $y[N] = x^t$. If $x = 0$ there is no soundness guarantee but solver can at least provide solutions with $y[N] = 0$ and $y[N] = 1$.

Proof. For $x = 0$:

- to output 0: set all $y[i]$ to 0
- to output 1:
 1. find the last index i such that $\text{bits}[N-1-i] = 0$
 2. for all $j \leq i$ set $y[j]$ to 0
 3. for all $j > i$ set $y[j]$ to 1

For $x \neq 0$ we will prove that for all $k \leq N$ statement $y[k] = x^{t_k}$ holds, where $t_k = 1 + 2^k + \sum_{i=0}^{k-1} \text{bits}[N-k+i]2^i$ for $k = 0$ last sum is 0.

Base: $y[0] = x^2$ and for empty bits array that t is $1 + 2^k = 1 + 2^0 = 2$.

Step: We prove that $y[k+1] = x^{t_{k+1}}$,

$$t_{k+1} = 1 + 2^{k+1} + 2(t_k - 1 - 2^k) + \text{bits}[N-k] = 2t_k - 1 + \text{bits}[N-k]$$

If $\text{bits}[N-k] = 1$

$$t_{k+1} = 2t_k - 1 + 1 = 2t_k$$

and we set $y[k+1]$ to $y[k]^2 = x^{2t_k} = x^{t_{k+1}}$

If $\text{bits}[N-k] = 0$

$$t_{k+1} = 2t_k - 1$$

and we set $y[k+1]$ to $\frac{y[k]^2}{x} = \frac{x^{2t_k}}{x} = x^{2t_k-1} = x^{t_{k+1}}$

□

Gadget 7: Certification by exponent**Functionality:**

Given a variable x prove that it is in the base field $\mathbb{F}$ (and certify it).

Variable ordering:

$\underline{x} \prec x_1$

Builder:

Use **Gadget 6** to constrain $x_1 \Leftarrow x^{\frac{q-1}{2}}$
 $x_1^2 = \mathbb{1} \cdot \mathbb{1}$

Lemma 13. This constraint system is solvable iff $x \in \mathbb{F}$.

Proof. For $x = 0$ solver can use the solution that gives $y[N] = \mathbb{1}$. For $x \neq 0, x \in \mathbb{F}$ solver has $y[N] = x^{\frac{q-1}{2}}$, $x_1 = x^{q-1}$ which is 1 iff $x \in \mathbb{F} \setminus \{0\}$. □

4.6 Pairing computation

We proceed with the pairing computation.

Algorithm 7 Miller loop on BN curves [AKL⁺11]

Input: $P \in \mathcal{G}_1, Q \in \mathcal{G}_2, m = |6u + 2| = \sum_{i=0}^{\lfloor \log(m) \rfloor} m_i 2^i$

Output: $\langle P, Q \rangle \in \mathcal{G}_t$

▷ *Miller loop:*

$T \leftarrow Q, f \leftarrow 1$

for $0 \leq j < \lfloor \log(m) \rfloor$ **do**

$i \leftarrow \lfloor \log(m) \rfloor - j - 1$

$f \leftarrow f^2 \cdot \ell_{T,T}(P), T \leftarrow 2T$

if $m_i = 1$ **then**

$f \leftarrow f \cdot \ell_{T,Q}(P), T \leftarrow T + Q$

end if

end for

$Q_1 \leftarrow \text{Frob}(Q)$

$Q_2 \leftarrow \text{Frob}^2(Q)$

if $u < 0$ **then**

$T \leftarrow -T$

$f \leftarrow f^{-1}$

end if

$f \leftarrow f \cdot \ell_{T,Q_1}(X), T \leftarrow T + Q_1$

$f \leftarrow f \cdot \ell_{T,-Q_2}(X), T \leftarrow T - Q_2$

▷ *This value f will be later denoted $\langle P, Q \rangle_{\text{Miller}}$*

▷ *Final exponent:*

$f \leftarrow f^{(q^{12}-1)/r}$

We will use the optimal Ate pairing algorithm on BN curves. The BN curve family is parametrized by a parameter u satisfying $r = 36u^4 + 36u^3 + 18u^2 + 6u + 1$. The extension degree is $d = 12$ and our base

field $\mathbb{F} = \mathbb{F}_q$. In the algorithm below, the notation $\ell_{A,B}$ means the equation of the line passing through the points A, B (normalized as $y + \star x + \star$). For simplicity, in what follows we will assume $u > 0$ (this holds for our target curve BN254 [BN06]).

For a fixed point Q , it is possible to precompute all the values of T for all the iterations of the loop. Moreover, the resulting value of f is a product of 2^i -powers of some collection of linear inhomogeneous polynomials in variables $(x, y) = P$ with coefficients in $\mathbb{F}_q^{12}$.

We will precompute and certify all the values of the form $u_i = xs_i$ for $i < 12$ and then will use a method similar to **Gadget 2** to compute the Miller loop. A linefunc term $\ell(x, y) = y + \ell_1 x + \ell_0$ will then take the form $y + \sum_i \ell_{1,i} u_i + \sum_i \ell_{0,i} s_i$.

Gadget 8: Multiplication terms of the Miller loop

Functionality:

Given a point $P = (x, y)$, checks that it is on a curve $y^2 = x^3 + b$ and both $x, y \in \mathbb{F}$. Outputs and certifies $u_i = xs_i$ for all $i < 12$.

Variable ordering:

Ordering is not important, but all outputs $\underline{u}_i$ are certified.

Builder:

Apply **Gadget 7** to x and y . Additionally extract from these gadgets the variables $x_2 = x^2, y_2 = y^2$.
 $(y_2 - b) \cdot \mathbb{1} = x_2 \cdot x$.

▷ *This validates that (x, y) is on the curve.* ◁

for $1 \leq i < 12$ **do**

$u_i \leftarrow xs_i$

$u_i \cdot \mathbb{1} = x \cdots s_i$

 ▷ *These constraints are not certifying yet* ◁

$R(x) \leftarrow x^{2i} \bmod E(x)$

$u_i^2 = x_2 \cdot \sum_{i=0}^{11} R_{(i)} s_i$

 ▷ *Certify using the same trick as for the extension elements themselves* ◁

end for

return u_i for $i < 12$

Comments in **Gadget 8** are self-explanatory.

In the Miller loop gadget, we will use the following notation: for a precomputed line function $\ell(x, y)$ we denote $\ell_{i,j} \in \mathbb{F}_p$ the $\mathbb{F}_p$ -coordinates of coefficients of ℓ :

$$\ell(x, y) = y + \ell_1 x + \ell_0 = y + \sum \ell_{1,j} u_j + \sum \ell_{0,j} s_j$$

Algorithm 8 Precomputation for the reverse Miller loop gadget

Input: Point $Q \in \mathcal{G}_2$
Output: Coefficients of all $\ell_i, \ell'_i, \ell_A, \ell_B$.
 $T \leftarrow Q, f \leftarrow 1$
for $0 \leq j < \lfloor \log(m) \rfloor$ **do**

$Q_1 \leftarrow \text{Frob}(Q)$
$Q_2 \leftarrow \text{Frob}^2(Q)$
$\ell_L \leftarrow \ell_{T, Q_1}$
$T \leftarrow T + Q_1$
$\ell_R \leftarrow \ell_{T, -Q_2}$
$T \leftarrow T - Q_2$

$i \leftarrow \lfloor \log(m) \rfloor - j - 1$
 $\ell_i \leftarrow \ell_{T, T}, T \leftarrow 2T$
if $m_i = 1$ **then**

$\ell'_i \leftarrow \ell_{T, Q}, T \leftarrow T + Q$

end if

end for

We also need a gadget that outputs the twists from $\mu_{2^k} = \mu_{2^\infty} \cap \mathcal{K}$ to move everything into G_{odd} . In our target curve BN254 [BN06], $k = 5$.

Gadget 9: TwistCheck - Twist check
Functionality:

Checks that $x \in \mu_{2^k}$.

Variable ordering:

Not important, and everything is certified.

Builder:

```

 $x_1 \leftarrow x$ 
for  $1 \leq i < k - 1$  do
   $x_{i+1} \leftarrow x_i^2$ 
   $x_{i+1} \cdot \mathbb{1} = x_i^2$ 
end for
 $x_{k-1}^2 = \mathbb{1} \cdot \mathbb{1}$ 

```

We will denote finding a cofactor twist and applying **Gadget 9** by $\leftarrow \mu_{2^k}$. The suggested solver value of the twist is inferred from the context (it is instantly multiplied by a value in order to put the result in an odd-order subgroup).

Gadget 10: OddGroupMillerLoop - Miller loop with values in G_{odd}
Functionality:

Given the output data of **Gadget 8** corresponding to an input point $P \in \mathcal{G}_1$:

(Soundness:) Outputs the value $f \in \overline{\mathbb{F}}$ such that

$$f^{2^{(\lfloor \log m \rfloor + 2)}} / \langle P, Q \rangle_{\text{Miller}} \in \mu_{2^k}$$

(Completeness:) Solver can choose $f \in G_{\text{odd}}$ to be

$$f = (\langle P, Q \rangle_{\text{Miller}} \cdot m)^{2^{-(\lfloor \log m \rfloor + 2)}}$$

where $m \in \mu_{2^k}$.

Variable ordering:

This gadget only uses output-certifying gadgets as the components, so it has the same behavior: inputs must be certified, outputs are certified automatically.

Builder:


```


$$a_L \Leftarrow y + \sum_{r=0}^{11} ((\ell_L)_{1,r} u_r + (\ell_L)_{0,r} s_r)$$


$$t_L \Leftarrow \mu_{2^k}$$


$$b_L \Leftarrow \sqrt{a_L t_L}$$


$$a_R \Leftarrow y + \sum_{r=0}^{11} ((\ell_R)_{1,r} u_r + (\ell_R)_{0,r} s_r)$$


$$t_R \Leftarrow \mu_{2^k}$$


$$b_R \Leftarrow \sqrt{a_R t_R}$$


$$f[0] \Leftarrow \sqrt{b_L b_R}$$

 $\triangleright$  This value contains the contributions from two last exceptional multiplications taken to the power  $2^{-2}$ .  $\triangleleft$ 
for  $0 \leq i < \lfloor \log(m) \rfloor$  do
|    $a_1[i] \Leftarrow y + \sum_{r=0}^{11} ((\ell_i)_{1,r} u_r + (\ell_i)_{0,r} s_r)$ 
|    $t_1[i] \Leftarrow \mu_{2^k}$ 
|    $b_1[i] \Leftarrow \sqrt{a_1[i] t_1[i]}$ 
|   if  $m_i = 1$  then
|   |    $a_2[i] \Leftarrow y + \sum_{r=0}^{11} ((\ell'_i)_{1,r} u_r + (\ell'_i)_{0,r} s_r)$ 
|   |    $t_2[i] \Leftarrow \mu_{2^k}$ 
|   |    $b_2[i] \Leftarrow \sqrt{a_2[i] t_2[i]}$ 
|   else
|   |    $b_2[i] \Leftarrow 1$ 
|   end if
|    $s[i] \Leftarrow \sqrt{b_1[i] b_2[i]}$ 
|    $\triangleright$  This value contains contribution of  $i$ -th iteration of the Miller loop, with power  $2^{-2}$   $\triangleleft$ 
|    $f[i+1] \Leftarrow \sqrt{f[i] \cdot s[i]}$ 
end for
 $f \Leftarrow f[\lfloor \log(m) \rfloor]$ 
return  $f$ 

```

Lemma 14. OddGroupMillerLoop gadget has stated soundness and completeness properties.

Proof. Conceptually, the idea is similar to the OddGroupFixpowExp gadget. To keep track of the exponent, pretend that the computation is done in the group $\overline{\mathbb{F}}/\mu_{2^\infty}$ (so we are computing up to the cofactor μ_{2^∞}). Then, all the twist terms t_L, t_R, t_i can be ignored.

It now is easy to see that $f[0] = (\ell_L(P)\ell_R(P))^{2^{-2}}$. Similarly, $s[j] = (\ell_j(P)\ell'_j(P))^{2^{-2}}$. The total value that we obtain is

$$\begin{aligned}
 f &= \left((\ell_L(P)\ell_R(P)) \cdot \prod_{j=0}^{\lfloor \log(m) \rfloor - 1} (\ell_j(P)\ell'_j(P))^{2^j} \right)^{2^{-2-\lfloor \log(m) \rfloor}} = \\
 &= (\langle P, Q \rangle_{\text{Miller}})^{2^{-2-\lfloor \log(m) \rfloor}}
 \end{aligned}$$

To see the completeness property, choose each twist $(t_L, t_R, t_i\text{-s})$ in such a way that the product with corresponding a -term is in G_{odd} . This allows to maintain invariant that all computed square roots are also in G_{odd} . $\square$

Theorem 4. Given input points $\mathcal{P}, \mathcal{Q} \in \mathcal{G}_1 \subset \mathbb{F} \times \mathbb{F}$ and values $p_0, p_1, p_2, p_3, v_Z, z^4 \in \mathbb{F}_r$ represented as sequences of bits, the pairing check

$$\langle \mathcal{P}, [\gamma^{-1}]_2 \rangle + [\alpha_Z Z(\tau)]_t = \langle \mathcal{Q}, [\tau]_2 \rangle - z \cdot \langle \mathcal{Q}, [1]_2 \rangle + \sum_U v_U [\alpha_U]_t$$

can be performed by a projectively safe constraint system over $\mathbb{F}$ using the gadgets defined above.

Proof. Slightly rearrange terms:

$$\langle \mathcal{P}, [\gamma^{-1}]_2 \rangle + \langle \mathcal{Q}, [-\tau]_2 \rangle + z \cdot \langle \mathcal{Q}, [1]_2 \rangle + \sum_U v_U [\alpha_U]_t = [-\alpha_Z Z(\tau)]_t$$

Gadget 11: SNARK.VerifyPairingEq

Builder:

```

... s_i ... ← FindExtensionBasis()
▷ Gadget 4.
For both  $\mathcal{P}$  and  $\mathcal{Q}$ , call the Gadget 8.
▷ Call Gadget 10 thrice (with different precomputed constants) to compute Miller loop results up to  $\mu_{2^k}$ -cofactor:
S1 ← OddGroupMillerLoop( $\mathcal{P}, [\gamma^{-1}]_2$ )
S2 ← OddGroupMillerLoop( $\mathcal{Q}, [-\tau^4]_2$ )
▷ Pairing  $\langle \mathcal{Q}, [1]_2 \rangle$  is shifted to compensate for exp application
▷ Value of z can be truncated up to  $\lambda = 128$  bits, denote  $B_z = \lambda + 2$ 
S'3 ← OddGroupMillerLoop( $\mathcal{Q}, [2^{B_z+2}]_2$ )
S3 ← OddGroupVarpowExp(S'3, z)
Smillier ←  $\sqrt{\sqrt{S_1 \cdot S_2} \cdot \sqrt{S_3} \cdot \mathbb{1}}$ 
▷ Apply Gadget 2 to compute final exponentiation:
Spair ← OddGroupFixpowExp(Smillier,  $(q^{12} - 1)/r$ ).
▷ This result is shifted by  $2^{-\sigma}$  where  $\sigma = (\lfloor \log m \rfloor + 2) + 2 + (\lfloor \log(\frac{q^{12}-1}{r}) \rfloor + 1)$ 
Get constants  $[-2^\kappa \alpha_U]_t$  for  $U \in \{A, C, H, Z\}$  where  $\kappa = -\sigma + \lfloor \log r \rfloor + 3$ .
SU ← OddGroupVarpowExp( $[-2^\kappa \alpha_U]_t, p_i$ ) for  $U \in \{A, C, H, Z\}$ 
Svals ←  $\sqrt{\sqrt{S_A S_C} \cdot \sqrt{S_H S_Z}}$ 
Slhs ←  $\sqrt{S_{\text{pair}} S_{\text{vals}}}$ 
Srhs ←  $[-2^{-\sigma-1} \alpha_Z Z(\tau)]_t$ 
Slhs ·  $\mathbb{1} = S_{\text{rhs}} \cdot \mathbb{1}$ 

```

This gadget has the following soundness and completeness properties: it correctly computes the pairing representative in the group $\overline{\mathbb{F}}^*/\mu_{2^\infty}$ and checks that it is equal to 1. The solver can always choose a representative in the group G_{odd} . This is implied by the respective properties of all individual gadgets used in this one. $\square$

4.7 Poseidon hash

Poseidon hash can be instantiated with $x \mapsto x^5$ S-box for the base field of BN254.

Gadget 12: Poseidon S-box

Functionality:

Given a variable x construct a variable $x_5 = x^5$

Variable ordering: $\underline{x} \prec x_5$ **Builder:**

$$x^2 = 1 \cdot x_2$$

$$x_2 \leftarrow x^2$$

$$1 \cdot x_3 = x_2 \cdot x$$

$$x_3 \leftarrow x_2 \cdot x$$

$$1 \cdot x_5 = x_2 \cdot x_3$$

$$x_5 \leftarrow x_2 \cdot x_3$$

▷ *At this point, everything is constrained but nothing is certified.*

<

$$x_2^2 = x \cdot x_3$$

$$x_3^2 = x \cdot x_5$$

Gadget 13: Poseidon permutation**Functionality:**

Given a state $(t_0, \dots, t_n)$ construct $(t'_0, \dots, t'_n) = \text{PoseidonPermutation}(t_0, \dots, t_n)$.

Variable ordering: $t_i \prec t'_i$ **Builder:**

Apply [Gadget 12](#) and linear layers according to the Poseidon hash algorithm. Each S-box gadget certifies its inputs and linear operations are free, so in effect it certifies everything but the outputs.

One important thing is that the outputs of this Poseidon permutation gadget are left uncertified, so if they are dropped they need to be certified separately.

4.8 Non-native arithmetic

The purpose of this section is to perform the check $v_A^2 - v_C = v_H v_Z + u$ of [Algorithm 6](#) (SNARK.Verify). It is best done in the exponent. Denote $B_r = \lfloor \log(r) \rfloor + 2$.

Gadget 14: SNARK.VerifyArith**Builder:**

Input: v_A, v_C, v_Z, v_H, u (all bit sequences representing elements of $\mathbb{F}_r$)

$$[2^{B_r+1}v_A]_t \leftarrow \text{OddGroupVarpowExp}([2^{2B_r+1}]_t, v_A)$$

$$[2v_A^2]_t \leftarrow \text{OddGroupVarpowExp}([2^{B_r+1}v_A]_t, v_A)$$

$$[-2v_C]_t \leftarrow \text{OddGroupVarpowExp}([-2^{B_r+1}]_t, v_C)$$

▷ *This is a normal sqrt gate, just in the additive notation:*

<

$$[v_A^2 - v_C]_t \leftarrow \frac{1}{2}([2v_A^2]_t + [-2v_C]_t)$$

$$[2^{B_r+1}v_Z]_t \leftarrow \text{OddGroupVarpowExp}([2^{2B_r+1}]_t, v_Z)$$

$$[2v_H v_Z]_t \leftarrow \text{OddGroupVarpowExp}([2^{B_r+1}v_Z]_t, v_H)$$

$$[2u]_t \leftarrow \text{OddGroupVarpowExp}([2^{B_r+1}]_t, u)$$

$$[v_H v_Z + u]_t \leftarrow \frac{1}{2}([2v_H v_Z]_t + [2u]_t)$$

$$[v_A^2 - v_C]_t = [v_H v_Z + u]_t$$

Note 6. **Gadget 14** also operates in G_{odd} and $\mathbb{F}^*/\mu_{2^\infty}$.

4.9 Final gadget

Gadget 15: SNARK.VerifyFull

Builder:

Input: Proof = $(u, \mathcal{P}, \mathcal{Q}, v_U$ for $U \in \{A, C, H, Z\}$)
 $z \leftarrow \text{Hash}(\text{pp}, \mathcal{P}, u)[0..\lambda]$
Bitcheck: u, v_U -s, all outputs of the Poseidon permutation.
 $\dots s_i \dots \leftarrow \text{FindExtensionBasis}()$
 SNARK.VerifyArith(u, v_U)
 SNARK.VerifyPairingEq($\mathcal{P}, \mathcal{Q}, v_U$)

4.10 Cost estimates

A python script¹ (approximately) modeling the gadgets above gives the following results (with v denoting variable count and g denoting gate count):

Subroutine	Variables	Gates
Extension	11	22
Glue code	1265	1270
Hash	1017	1192
Arithmetic	3578	3580
Pairing equation	8212	8307
Totals	14083	14371

Substituting the ciphertext size formula $v(2g + 1)^2|\mathbb{F}|$, we get 338 TB size of the ciphertext.

These numbers should not be fully trusted - after all, we do not have a full implementation and only a modeling script which could very well miss some important subroutine completely. On the other hand, due to the cubic scaling of the ciphertext size even marginal improvements in the circuit size translate into very significant improvements in the ciphertext size.

Size of the ciphertext might possibly be decreased further - for example, by picking a better hash with a certification-friendly S-box or choosing a curve with smaller 2-adicity of $\mathbb{F}^d$.

Overall, we conclude that this approach to witness encryption, while very costly, is realistically implementable.

References

- [AIK07] B. Applebaum, Y. Ishai, and E. Kushilevitz. Cryptography with Constant Input Locality. In A. Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 92–110. Springer, Berlin, Heidelberg, August 2007 (Cited on page 3).
- [AK14] P. Austrin and S. Khot. A Simple Deterministic Reduction for the Gap Minimum Distance of Code Problem. *IEEE Transactions on Information Theory*, 60(10):6636–6645, 2014 (Cited on page 4).

¹ <https://github.com/alloc-init/adp-estimates>

- [AKL⁺11] D. F. Aranha, K. Karabina, P. Longa, C. H. Gebotys, and J. C. López-Hernández. Faster Explicit Formulas for Computing Pairings over Ordinary Curves. In K. G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 48–68. Springer, Berlin, Heidelberg, May 2011 (Cited on page 29).
- [AMY⁺25] S. Agrawal, A. Modi, A. Yadav, and S. Yamada. Evasive LWE: Attacks, Variants & Obfuscopia. Cryptology ePrint Archive, Report 2025/375, 2025 (Cited on page 4).
- [BC16] O. Blazy and C. Chevalier. Structure-Preserving Smooth Projective Hashing. In J. H. Cheon and T. Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 339–369. Springer, Berlin, Heidelberg, December 2016 (Cited on page 5).
- [BGI⁺01] B. Barak, O. Goldreich, R. Impagliazzo, S. Rudich, A. Sahai, S. P. Vadhan, and K. Yang. On the (Im)possibility of Obfuscating Programs. In J. Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, August 2001 (Cited on page 3).
- [BIJ⁺20] J. Bartusek, Y. Ishai, A. Jain, F. Ma, A. Sahai, and M. Zhandry. Affine Determinant Programs: A Framework for Obfuscation and Witness Encryption. In T. Vidick, editor, *ITCS 2020*, volume 151, 82:1–82:39. LIPIcs, January 2020 (Cited on pages 3, 4, 5, 6, 8, 9, 10, 11, 12, 13, 14, and 16).
- [BIM⁺23] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. B. Melé. Circom: A Circuit Description Language for Building Zero-Knowledge Applications. *IEEE Trans. Dependable Secur. Comput.*, 20(6):4733–4751, 2023 (Cited on page 17).
- [BIO⁺20] O. Barta, Y. Ishai, R. Ostrovsky, and D. J. Wu. On Succinct Arguments and Witness Encryption from Groups. In D. Micciancio and T. Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 776–806. Springer, Cham, August 2020 (Cited on page 4).
- [BL20] F. Benhamouda and H. Lin. Mr NISC: Multiparty Reusable Non-Interactive Secure Computation. In R. Pass and K. Pietrzak, editors, *TCC 2020, Part II*, volume 12551 of *LNCS*, pages 349–378. Springer, Cham, November 2020 (Cited on page 5).
- [BN06] P. S. L. M. Barreto and M. Naehrig. Pairing-Friendly Elliptic Curves of Prime Order. In B. Preenel and S. Tavares, editors, *SAC 2005*, volume 3897 of *LNCS*, pages 319–331. Springer, Berlin, Heidelberg, August 2006 (Cited on pages 16, 21, 30, and 31).
- [BÜW24] C. Brzuska, A. Ünal, and I. K. Y. Woo. Evasive LWE Assumptions: Definitions, Classes, and Counterexamples. In K.-M. Chung and Y. Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 418–449. Springer, Singapore, December 2024 (Cited on page 4).
- [CFK24] M. Campanelli, D. Fiore, and H. Khoshakhlagh. Witness Encryption for Succinct Functional Commitments and Applications. In Q. Tang and V. Teague, editors, *PKC 2024, Part II*, volume 14602 of *LNCS*, pages 132–167. Springer, Cham, April 2024 (Cited on page 5).
- [DJM⁺25] N. Döttling, A. Jain, G. Malavolta, S. Mathialagan, and V. Vaikuntanathan. Simple and General Counterexamples for Private-Coin Evasive LWE. In Y. T. Kalai and S. F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 73–92. Springer, Cham, August 2025 (Cited on page 4).
- [DMS24] M. Dellepère, P. Mishra, and A. Shirzad. Garuda and Pari: Faster and Smaller SNARKs via Equificient Polynomial Commitments. Cryptology ePrint Archive, Report 2024/1245, 2024 (Cited on pages 4 and 22).
- [FHAS24] N. Fleischhacker, M. Hall-Andersen, and M. Simkin. Extractable Witness Encryption for KZG Commitments and Efficient Laconic OT. In K.-M. Chung and Y. Sasaki, editors, *ASIACRYPT 2024, Part II*, volume 15485 of *LNCS*, pages 423–453. Springer, Singapore, December 2024 (Cited on page 5).
- [GGH15] C. Gentry, S. Gorbunov, and S. Halevi. Graph-Induced Multilinear Maps from Lattices. In Y. Dodis and J. B. Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 498–527. Springer, Berlin, Heidelberg, March 2015 (Cited on page 4).
- [GGH⁺13] S. Garg, C. Gentry, S. Halevi, M. Raykova, A. Sahai, and B. Waters. Candidate Indistinguishability Obfuscation and Functional Encryption for all Circuits. In *54th FOCS*, pages 40–49. IEEE Computer Society Press, October 2013 (Cited on pages 3 and 4).
- [GGS⁺13] S. Garg, C. Gentry, A. Sahai, and B. Waters. Witness encryption and its applications. In D. Boneh, T. Roughgarden, and J. Feigenbaum, editors, *45th ACM STOC*, pages 467–476. ACM Press, June 2013 (Cited on pages 3, 4, and 10).

- [GHK⁺25] S. Garg, M. Hajiabadi, D. Kolonelos, A. Kothapalli, and G.-V. Policharla. A Framework for Witness Encryption from Linearly Verifiable SNARKs and Applications. In Y. T. Kalai and S. F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 504–539. Springer, Cham, August 2025 (Cited on page 5).
- [GKP⁺24] S. Garg, D. Kolonelos, G.-V. Policharla, and M. Wang. Threshold Encryption with Silent Setup. In L. Reyzin and D. Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 352–386. Springer, Cham, August 2024 (Cited on page 5).
- [GKR⁺19] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. Cryptology ePrint Archive, Paper 2019/458, 2019 (Cited on page 21).
- [GLW14] C. Gentry, A. B. Lewko, and B. Waters. Witness Encryption from Instance Independent Assumptions. In J. A. Garay and R. Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 426–443. Springer, Berlin, Heidelberg, August 2014 (Cited on page 3).
- [GMM17] S. Garg, M. Mahmoody, and A. Mohammed. Lower Bounds on Obfuscation from All-or-Nothing Encryption Primitives. In J. Katz and H. Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 661–695. Springer, Cham, August 2017 (Cited on page 4).
- [Gro16] J. Groth. On the Size of Pairing-Based Non-interactive Arguments. In M. Fischlin and J.-S. Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016 (Cited on page 5).
- [HJL25] Y.-C. Hsieh, A. Jain, and H. Lin. Lattice-Based Post-quantum iO from Circular Security with Random Opening Assumption. In Y. T. Kalai and S. F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 3–38. Springer, Cham, August 2025 (Cited on page 4).
- [IK00] Y. Ishai and E. Kushilevitz. Randomizing Polynomials: A New Representation with Applications to Round-Efficient Secure Computation. In *41st FOCS*, pages 294–304. IEEE Computer Society Press, November 2000 (Cited on page 3).
- [IK02] Y. Ishai and E. Kushilevitz. Perfect Constant-Round Secure Computation via Perfect Randomizing Polynomials. In P. Widmayer, F. T. Ruiz, R. M. Bueno, M. Hennessy, S. Eidenbenz, and R. Conejo, editors, *ICALP 2002*, volume 2380 of *LNCS*, pages 244–256. Springer, Berlin, Heidelberg, July 2002 (Cited on page 3).
- [IK97] Y. Ishai and E. Kushilevitz. Protecting Data Privacy in Private Information Retrieval Schemes. Cryptology ePrint Archive, Report 1997/006, 1997 (Cited on page 3).
- [JLS21] A. Jain, H. Lin, and A. Sahai. Indistinguishability obfuscation from well-founded assumptions. In S. Khuller and V. V. Williams, editors, *53rd ACM STOC*, pages 60–73. ACM Press, June 2021 (Cited on page 4).
- [JLS22] A. Jain, H. Lin, and A. Sahai. Indistinguishability Obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0 . In O. Dunkelman and S. Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 670–699. Springer, Cham, 2022 (Cited on page 4).
- [KS99] A. Kipnis and A. Shamir. Cryptanalysis of the HFE Public Key Cryptosystem by Relinearization. In M. J. Wiener, editor, *CRYPTO ’99*, volume 1666 of *LNCS*, pages 19–30. Springer, Berlin, Heidelberg, August 1999 (Cited on pages 14 and 16).
- [KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. Constant-Size Commitments to Polynomials and Their Applications. In M. Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010 (Cited on page 22).
- [RVV24] S. Ragavan, N. Vafa, and V. Vaikuntanathan. Indistinguishability Obfuscation from Bilinear Maps and LPN Variants. In E. Boyle and M. Mahmoody, editors, *TCC 2024, Part IV*, volume 15367 of *LNCS*, pages 3–36. Springer, Cham, December 2024 (Cited on page 4).
- [Tsa22] R. Tsabary. Candidate Witness Encryption from Lattice Techniques. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 535–559. Springer, Cham, August 2022 (Cited on pages 3 and 4).
- [VWW22] V. Vaikuntanathan, H. Wee, and D. Wichs. Witness Encryption and Null-IO from Evasive LWE. In S. Agrawal and D. Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 195–221. Springer, Cham, December 2022 (Cited on pages 3 and 4).

- [Wee22] H. Wee. Optimal Broadcast Encryption and CP-ABE from Evasive Lattice Assumptions. In O. Dunkelman and S. Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 217–241. Springer, Cham, 2022 (Cited on page 3).